

The POSIX port

**μOS++ III SMP on the development machine: one host thread
per CPU**

Port: micro-os-plus-iii-posix-arch

The POSIX port

How `micro-os-plus-iii-posix-arch` runs $\mu\text{OS++ III SMP}$ on the development machine: what "a CPU" means when there is no silicon, how a context switch works when there is no assembly, and what the port found in the kernel on its first day.

This is the fourth architecture project, beside `-aarch32`, `-aarch64` and `-cortexm`. It is the only one that is not a migration. The other three carried code that had already been brought up on hardware; this one is new work, written against decision **D12** of the unification design — `posix-arch` must be SMP, modelled as one host thread per CPU.

It is also the only port whose machine can be changed with a command-line switch, which is what makes it worth having: `-DNCPU=8` on a four-core laptop is an oversubscribed eight-core SoC, and races that need a particular interleaving to appear will appear.

Contents

1. [What it is](#)
2. [The machine model](#)
3. [The port contract, symbol by symbol](#)
4. [Boot: from `main\(\)` to the first thread](#)
5. [Threads](#)
6. [The scheduler](#)
7. [The context switch and the deferred publish](#)
8. [Interrupts](#)
9. [Preemption](#)
10. [Time](#)
11. [Memory](#)
12. [Faults and diagnostics](#)
13. [The board](#)
14. [The build](#)
15. [The tests](#)
16. [Running them](#)
17. `OS_NCPU=1`
18. [The defect this port found](#)
19. [Tips, tricks and traps](#)
20. [Honest limits](#)
21. [Decisions, and what was rejected](#)
22. [Reference tables](#)
23. [Worked examples](#)

1. What it is

```
micro-os-plus-iii-posix-arch/
├─ CMakeLists.txt                board discovery, the port target
├─ include/
│   └─ cmsis-plus/rtos/port/
│       └─ os-c-decls.h          93 lines    types; the thread context
│       └─ os-decls.h           191 lines    the C++ port contract
│       └─ os-inlines.h         252 lines    CPU id, masking, the locks
│       └─ host_cpu.hpp          97 lines    the CPU model's interface
│       └─ exception_handler.hpp 29 lines
│       └─ hw_result.hpp         53 lines    the verdict, as an exit status
├─ src/
│   └─ rtos/os-core.cpp          597 lines  the port's half of the scheduler
│   └─ host_cpu.cpp              413 lines  host threads as CPUs: tick, IPI
│   └─ exception_handler.cpp     227 lines  SIGSEGV/BUS/FPE/ILL, reported
│   └─ free-store.cpp            133 lines  the application heap
│   └─ board-contract.cpp        43 lines  no code; `#error` if a board lies
│   └─ diag/trace-posix.cpp      80 lines  carried from upstream v1.0.1
└─ test/
    └─ CMakeLists.txt            one loop over test/<board>/*/
    └─ run.sh                    dispatcher; BOARD picks the board
    └─ boards/native/
        └─ board.cmake            every fact about this board
        └─ include/{uart,led,timer_arm,smp}.hpp
        └─ src/{heap,smp}.cpp
        └─ run.sh
    └─ native/                    this board's 10 applications
        └─ include/{test-console,test-smp-boot}.hpp
        └─ src/test-smp-boot.cpp  shared support, one copy
        └─ tests.cmake            the knobs a listing cannot express
        └─ smp_test0 ... smp_test4
        └─ smp-mat-test smp-num-test smp-pipeline-test smp-pro-cons-test
        └─ mutex-stress           the OS_NCPU=1 leg
```

Roughly 2,300 lines of port and board code. One board. Ten test applications.

board	"silicon"	CPUs	tests	verdict
native	the host kernel	4, -DNCPU=	10	10 passed / 0 skipped / 0 failed

The port depends on the kernel (`micro-os-plus-iii-smp`) and the device layer (`micro-os-plus-iii-devices`), holds no copy of either, and neither of them knows it exists. That one-way dependency is why the kernel tree stays merge-clean against upstream.

Where it came from

Upstream `micro-os-plus-iii-posix-arch` v1.0.1 is 19 files and is **not** SMP. Its `NOTES.md` states the limitation plainly:

the scheduler runs in cooperative mode only; thread pre-emption might be possible, but was considered not worth the effort.

Upstream's model: one host thread; μ OS++ threads are `ucontext` coroutines switched with `getcontext`/`makecontext`/`swapcontext`; the tick is `setitimer(ITIMER_REAL)` raising `SIGALRM`; "interrupt disable" is `sigprocmask` on that signal.

What was kept: the `ucontext` machinery, `trace-posix.cpp`, and the macro block in `os-c-decls.h` that lets the port build against libucontext instead of glibc's. Everything else is new.

2. The machine model

A host thread is a CPU. A `ucontext` is a μ OS++ thread.

Two levels, and keeping them straight is the whole discipline of this port:

```
process
├─ host thread 0 — CPU 0 — runs one  $\mu$ OS++ thread at a time
├─ host thread 1 — CPU 1 — "
├─ host thread 2 — CPU 2 — "
└─ host thread 3 — CPU 3 — "

 $\mu$ OS++ threads: main, idle, idle1..3, w0..w7, ...
                each one a ucontext + a stack
                free to move between CPUs at every switch
```

`OS_NCPU` host threads are created at startup and are never destroyed. Each runs the scheduler and *is*, for everything the kernel can observe, a core: it has its own interrupt mask (its signal mask), its own tick (its own `timer_create` timer), its own handler-mode flag, and its own entries in `lock_state[]` and `_port_ctx_pending[]`.

μ OS++ threads remain `ucontext` contexts switched **within** a CPU — upstream's machinery, preserved deliberately. What is new, and what makes this port hard, is that *a context saved by one CPU may be resumed by another*. That is a data race unless the handover is ordered; §7 is about nothing else.

The mapping, concern by concern

concern	AArch64	AArch32	Cortex-M	here
CPU id	<code>MRS MPIDR_EL1</code> & 3	<code>MRC MPIDR</code> & 3	<code>SIO_CPUID</code>	<code>thread_local</code> <code>unsigned</code> <code>_this_cpu</code>
interrupt mask	<code>MSR DAIFSET/CLR</code> , #2	<code>CPSID i</code> / <code>CPSIE i</code>	<code>PRIMASK</code> / <code>BASEPRI</code>	<code>pthread_sigmask(SI</code> <code>G_BLOCK, irq_set)</code>
interrupt state	<code>MRS DAIF</code>	<code>MRS CPSR</code>	<code>PRIMASK</code>	<code>sigismember(old,</code> <code>SIGRTMIN)</code>
per-CPU tick	generic timer PPI	generic timer PPI	SysTick	<code>timer_create</code> + <code>SIGEV_THREAD_ID</code>
IPI	GIC SGI	GIC SGI / mailbox	SIO FIFO IRQ 25	<code>pthread_kill(tid,</code> <code>SIGRTMIN+1)</code>
kernel lock	<code>LDAXR</code> / <code>STLXR</code>	<code>LDREX</code> / <code>STREX</code>	SIO spinlock 0	<code>__atomic_exchange_</code> <code>n</code>
barrier	<code>DMB ISH</code>	<code>DMB</code>	<code>DMB</code>	<code>__atomic_*</code> seq-cst
in handler	<code>_in_isr[OS_NCPU]</code>	<code>_in_isr[OS_NCPU]</code>	<code>IPSR</code>	<code>_in_isr[OS_NCPU]</code> — identical
context switch	asm save/restore	asm save/restore	PendSV	<code>swapcontext</code>
idle "WFI"	<code>DSB SY; WFI</code>	<code>DSB; WFI</code>	<code>__DSB(); __WFI()</code>	<code>sigsuspend</code>
secondary release	spin table	spin table / mailbox	<code>launch_core1()</code>	<code>pthread_create</code>
interrupt stack	separate (<code>OS_HAS_INTERRUPTS_STACK</code>)	separate	MSP	none — the thread's own

The last row is a real difference, not an omission. The tick handler runs on the `μOS++` thread's own stack, deliberately, so that the signal frame is part of the context `swapcontext()` saves and therefore migrates with the thread. A separate interrupt stack belongs to the *host* thread and would be left behind. That single decision is why `OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES` is 32 KiB here and 2 KiB on the ARM ports.

The three defects D12 names

D12 lists three things that break the moment one host thread becomes several. All three are fixed, and the fixes are not workarounds — two of them make the abstraction *better*.

1. `sigprocmask` → `pthread_sigmask`. In a multithreaded process the behaviour of `sigprocmask` is unspecified (POSIX.1-2017; glibc documents it as such). Upstream used it throughout because upstream had exactly one thread. The replacement is the better abstraction: **a per-thread signal mask is per-CPU interrupt masking**, which is what a critical section has always meant.

2. `setitimer(ITIMER_REAL)` → `timer_create` + `SIGEV_THREAD_ID`. `ITIMER_REAL` delivers `SIGALRM` to an *arbitrary* thread of the process, so with N CPUs the tick lands on whichever host thread the kernel

happened to choose: the preemption of CPU 2 could be charged to CPU 0, and a tick would no longer be a per-CPU event. A per-thread timer is what a per-core timer actually is.

The signal is `SIGRTMIN`, not `SIGALRM`, because real-time signals *queue* rather than coalesce — a CPU that was masked for two tick periods sees two ticks. And the signal numbers are **functions**, not `constexpr` constants, because glibc's `SIGRTMIN` expands to a call to `__libc_current_sigrtmin()` (NPTL reserves the first few for itself):

```
namespace clock {
    inline int signal_number (void)    { return SIGRTMIN; }    // the tick
    inline int ipi_signal_number (void) { return SIGRTMIN + 1; } // the IPI
}
```

3. Native TLS and migration. `errno` and every `thread_local` are *host-thread* local, so a `µOS++` thread that migrates observes different storage after the switch than before it. D12 says this must be decided before migration is allowed, not after. The decision:

Migration is allowed. Native TLS is banned.

No port or application state may live in `errno` or a `thread_local` across a switch point. `errno` is read only inside the critical section that made the call that set it.

The one `thread_local` the port keeps is `_this_cpu`, and it is correct *by construction*: a host thread **is** a CPU, so storage private to a host thread is storage private to a CPU. It answers "where am I", never "what was I doing", and it is re-read after every switch rather than cached across one.

This was verified rather than assumed. The built image contains no `__tls_get_addr` — no general-dynamic TLS — and the accessor compiles to a single local-exec load with nothing held in a register:

```
000000000001d200 <port_cpu_id>:
    1d200:  64 8b 04 25 fc ff ff    mov    %fs:0xfffffffffffffc,%eax
    1d207:  ff
    1d208:  c3                    ret
```

One consequence of the same ban, in a place it is easy to miss: **the application free store is `µOS++`'s own `first_fit_top`, not glibc `malloc`**. A `µOS++` thread can be preempted inside an allocation and resumed on a different host thread, and glibc's arena lock would then be released by a thread that never took it. Nothing in the C library promises that works. See §11.

3. The port contract, symbol by symbol

The kernel's SMP patch is small — 379 lines — and asks a port for very little. This is the whole of it, and where this port answers each item.

3.1 `os-c-decls.h` — the width-dependent half

symbol	here
<code>os_port_thread_stack_element_t</code>	<code>uint64_t</code>
<code>os_port_clock_timestamp_t</code>	<code>uint64_t</code>
<code>os_port_clock_duration_t</code>	<code>uint32_t</code>
<code>os_port_scheduler_state_t</code>	<code>bool</code> — locked / unlocked
<code>os_port_irq_state_t</code>	<code>bool</code> — <i>true when this CPU's signals are blocked</i>
<code>os_port_thread_context_t</code>	<code>{ stack_ptr; ucontext; }</code>

The file opens with a hard stop:

```
#if !defined(_XOPEN_SOURCE)
#error This port requires defining _XOPEN_SOURCE=600L or 700L globally
#endif
```

`ucontext` is an X/Open interface and its declarations differ without the feature-test macro. Getting that wrong is a *miscompile*, not a link error, so the port refuses to build rather than produce one. `CMakeLists.txt` sets `_XOPEN_SOURCE=700` and `_GNU_SOURCE` on the interface target, so nothing downstream has to remember.

The thread context is the one structural change from upstream, and the field order is load-bearing:

```
typedef struct
{
    os_port_thread_stack_element_t* stack_ptr;    /* FIRST, and not cosmetic */
    os_impl_ucontext_t ucontext;
} os_port_thread_context_t;
```

Upstream's context was the `ucontext` alone — upstream had one CPU and therefore never had to ask whether a saved context was safe to resume. The SMP scheduler *does* ask, in the kernel, in a line no port may edit. §7 is that story.

3.2 `os-decls.h` — the C++ contract

This port deliberately does **not** link `micro-os-plus::port-smp-decls`. That target's own README names this port as the example that must keep its own declarations: the shared copy is written for ports whose state types are integers and whose thread context is a bare stack pointer, and here the interrupt state is a signal-mask bit, the scheduler state is a `bool`, and the context carries a ucontext.

What it declares instead, with the same names, the same types and the same volatility as the ARM copy — so the kernel cannot tell the two apart:

```
namespace interrupts {
    extern sigset_t irq_set;                // tick + IPI: "the interrupts"
    extern "C" volatile bool _in_isr[OS_NCPU]; // per-CPU handler-mode flag
}

namespace scheduler {
    extern volatile state_t lock_state[OS_NCPU]; // per-CPU scheduler lock

    struct smp_klock_t {                    // the recursive kernel lock
        volatile uint32_t lock;              // the lock word
        volatile uint32_t owner;             // owning CPU, or SMP_NO_OWNER
        volatile uint32_t depth;             // nesting depth
    };

    struct smp_tlock_t { volatile uint32_t lock; }; // the timer leaf lock

    extern smp_klock_t _smp_klock;
    extern smp_tlock_t _smp_tlock;
    extern volatile unsigned _port_ctx_pending[OS_NCPU];
}
```

and the stack sizes:

macro	value	why
<code>OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES</code>	32 KiB	a signal frame (ucontext + FPU state) lands on the thread stack
<code>OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES</code>	64 KiB	2 × min, as upstream
<code>OS_INTEGER_RTOS_MAIN_STACK_SIZE_BYTES</code>	64 KiB	
<code>OS_INTEGER_RTOS_IDLE_STACK_SIZE_BYTES</code>	64 KiB	the idle thread takes ticks too
<code>SMP_NO_OWNER</code>	<code>0xFFFFFFFF</code>	identical to the ARM ports

3.3 `os-inlines.h` — the hot path

Everything here is `always_inline`, because it is on the path of every critical section. The file's header comment carries the AArch64 mapping table so the two can be read side by side.

```
extern thread_local unsigned _this_cpu;

inline unsigned port_cpu_id_inline (void) { return _this_cpu; }

inline bool locked (void)
{ return lock_state[port_cpu_id_inline ()] != state::unlocked; }
```

The kernel lock is a **spin**, exactly as on ARM, and for the same reason: it is held for a handful of instructions and is taken from inside signal handlers, where a `pthread_mutex` would be a call into code that is not async-signal-safe.

```
inline void _smp_klock_raw_acquire (void)
{
    while (__atomic_exchange_n (&_smp_klock.lock, 1u, __ATOMIC_ACQUIRE) != 0u)
    {
#ifdef __x86_64__ || defined(__i386__)
        __builtin_ia32_pause ();           // PAUSE
#elif defined(__aarch64__) || defined(__arm__)
        __asm__ volatile ("yield" ::: "memory");
#endif
    }
}

inline void _smp_klock_raw_release (void)
{ __atomic_store_n (&_smp_klock.lock, 0u, __ATOMIC_RELEASE); }
```

The pause hint matters more here than on silicon: the host oversubscribes, so a spinning CPU can be a *scheduled* host thread starving the one that holds the lock. `PAUSE` / `YIELD` tells the hardware, and on some hosts the hypervisor, to stop trying so hard.

`port_tmr_lock` / `port_tmr_unlock` are the same shape over `_smp_tlock` — a separate leaf lock for the timer list, so that arming a timer does not have to take the whole kernel.

The critical section is the port's most-executed code and has exactly the same two steps, in the same order, as AArch64's:

```

inline rtos::interrupts::state_t critical_section::enter (void)
{
    sigset_t old;
    ::pthread_sigmask (SIG_BLOCK, &irq_set, &old);          // 1. mask this CPU

    const unsigned cpu = scheduler::port_cpu_id_inline ();
    if (scheduler::_smp_klock.owner != cpu)                  // 2. take the lock,
    {                                                         // recursively
        scheduler::_smp_klock_raw_acquire ();
        scheduler::_smp_klock.owner = cpu;
    }
    scheduler::_smp_klock.depth = scheduler::_smp_klock.depth + 1;

    return ::sigismember (&old, clock::signal_number ()) != 0; // prior state
}

inline void critical_section::exit (rtos::interrupts::state_t state)
{
    const unsigned cpu = scheduler::port_cpu_id_inline ();
    if (scheduler::_smp_klock.owner == cpu && scheduler::_smp_klock.depth > 0)
    {
        const uint32_t d = scheduler::_smp_klock.depth - 1;
        scheduler::_smp_klock.depth = d;
        if (d == 0)
        {
            scheduler::_smp_klock.owner = SMP_NO_OWNER;    // owner FIRST ...
            scheduler::_smp_klock_raw_release ();           // ... lock word LAST
        }
    }
    ::pthread_sigmask (state ? SIG_BLOCK : SIG_UNBLOCK, &irq_set, nullptr);
}

```

Read the release order carefully; §6.4 explains why reversing it is a total freeze.

Note what the returned `state_t` is: not a register image but *the single bit "was the tick already blocked"*. `exit()` restores exactly that. A nested critical section therefore leaves the mask blocked on exit, which is correct.

3.4 What the port must define out of line

symbol	file	note
<code>extern "C" unsigned port_cpu_id(void)</code>	<code>os-core.cpp</code>	the kernel calls this by name
<code>port::scheduler::greeting()</code>	<code>os-core.cpp</code>	printed by the kernel's <code>main()</code>
<code>port::scheduler::initialize()</code>	<code>os-core.cpp</code>	
<code>port::scheduler::locked(state_t)</code>	<code>os-core.cpp</code>	scheduler lock/unlock
<code>port::scheduler::start()</code>	<code>os-core.cpp</code>	never returns
<code>port::scheduler::reschedule()</code>	<code>os-core.cpp</code>	
<code>port::scheduler::switch_stacks(sp)</code>	<code>os-core.cpp</code>	a friend of <code>rtos::thread</code>
<code>port::scheduler::wait_for_interrupt()</code>	<code>os-core.cpp</code>	the idle "WFI"
<code>port::context::create(ctx, func, args)</code>	<code>os-core.cpp</code>	
<code>port::clock_systick::start()</code>	<code>os-core.cpp</code>	no-op; see §10
<code>port::clock_highres::*</code>	<code>os-core.cpp</code>	<code>CLOCK_MONOTONIC</code>
<code>extern "C" void port_smp_ipi(unsigned cpu)</code>	<code>host_cpu.cpp</code>	
<code>os_startup_initialize_free_store()</code>	<code>free-store.cpp</code>	kernel's copy is ARM-only
<code>os_rtos*_out_of_memory_hook()</code>	<code>free-store.cpp</code>	weak
<code>port_fatal_exception(...)</code>	<code>exception_handler.cpp</code>	

And one the *board* must define, checked by `board-contract.cpp` :

```
extern "C" volatile uint32_t g_core_stage[OS_NCPU]; /* 0 = down, 3 = running */
```

`board-contract.cpp` contains no code at all. It exists to turn a missing board fact into a compile error with a sentence attached:

```

#if !defined(OS_NCPU)
#error "The board does not set OS_NCPU."
#endif
#if !defined(PORT_GREETING)
#error "The board does not define PORT_GREETING; the test banners print it."
#endif
#if OS_NCPU > 16
#warning "OS_NCPU above 16 oversubscribes almost any host; expect the tick to drift."
#endif

```

4. Boot: from `main()` to the first thread

Upstream's `NOTES.md` states the rule that shapes this whole section: *"For portability reasons, execution starts in the `main()` function."* There is no reset vector, no startup assembly, no `.init_array` walk to arrange — the host has already done all of it.

Two entry styles coexist, and both work:

A. The kernel's weak `main()` (`src/rtos/os-main.cpp`), used by `mutex-stress`. It prints the version banner, calls `port::scheduler::greeting()`, calls `scheduler::initialize()`, creates the `main` thread around `os_main()`, creates the idle thread and calls `scheduler::start()`.

B. The test's own `main()`, used by all nine carried tests, because they were written for bare metal where the same code had to run before a C library was ready. Identical in substance:

```

int main (int, char*[])
{
    scheduler::initialize ();                // ← the port's startup happens here

    static thread::stack::element_t main_stack[8192];    // 64 KiB
    thread::attributes attr = thread::initializer;
    attr.th_stack_address    = main_stack;
    attr.th_stack_size_bytes = sizeof (main_stack);
    static thread main_thread { "main", custom_main_trampoline, nullptr, attr };
    os_main_thread = &main_thread;

    os_startup_create_thread_idle ();
    scheduler::start ();                    // never returns
}

```

`port::scheduler::initialize()` — and the bug that put the hardware hooks there

```
result_t initialize (void)
{
    ::sigemptyset (&interrupts::irq_set);                // 1
    ::sigaddset (&interrupts::irq_set, clock::signal_number ());
    ::sigaddset (&interrupts::irq_set, clock::ipi_signal_number ());

    for (unsigned c = 0; c < OS_NCPU; ++c)                // 2
    {
        lock_state[c] = state::init;
        _port_ctx_pending[c] = 0;
        interrupts::_in_isr[c] = false;
    }

    host_cpu::install_handlers ();                        // 3

    os_startup_initialize_hardware_early ();              // 4
    os_startup_initialize_hardware ();

    ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, nullptr); // 5
    return result::ok;
}
```

1. `irq_set` first, before anything can enter a critical section: every mask operation in the port names this set.
2. Per-CPU state cleared for all CPUs, by CPU 0, before any other CPU exists.
3. `install_handlers()` registers the tick and IPI handlers process-wide, records CPU 0's `pthread_t`, sets `g_core_stage[0] = 3` and installs CPU 0's fault stack.
4. **The application's hardware hooks** — and this line is here because of a real bug. On a bare-metal target the kernel's `src/startup/startup.cpp` calls these two before `main()`; every carried test relies on it, because `os_startup_initialize_hardware()` is where a test brings up its console, prints its banner, installs the free store and calls `exception::init()`. Here there is no such startup, and `startup.cpp` is `__ARM_EABI__`-guarded from end to end — so nothing called them at all. The symptom was not a missing banner but a *silent* one: the free store was never installed (the kernel's `malloc` resource stayed in place) and `exception::init()` never ran, so the first SMP fault reported **nothing whatsoever**. Deliberately *after* `install_handlers()`, because `exception::init()` wants the alternate signal stack that call sets up.
5. Interrupts masked until `start()`, as every port does.

`port::scheduler::start()`

```
[[noreturn]] void start (void)
{
    ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, nullptr);

    static os_thread_t fake_thread[OS_NCPU];          // something to switch AWAY from
    const unsigned cpu = port_cpu_id ();
    memset (&fake_thread[cpu], 0, sizeof (os_thread_t));
    fake_thread[cpu].name = "fake_thread";
    host_cpu::current_thread (cpu) = (rtos::thread*) &fake_thread[cpu];

    for (unsigned c = 0; c < OS_NCPU; ++c) lock_state[c] = state::init;

#ifdef OS_USE_SMP_SCHEDULER
    rtos::scheduler::os_idle_thread_core[0] = ::os_idle_thread;
#endif

    host_cpu::start_this_cpu_tick ();
    reschedule ();                                // does not come back
    ...
}
```

The **fake thread** deserves a word. The very first switch has to save something, and this CPU's host-thread context is not a μ OS++ thread. The ARM ports use a zeroed `fake_thread` for exactly the same reason. It is never scheduled again, so it needs no stack; upstream's single-core port avoided it by using `setcontext` instead of `swapcontext`, which this port cannot do because `switch_stacks()` must be uniform for every switch.

CPU 0's idle thread is the kernel's `os_idle_thread`. The secondaries' are created by the *board's* test support (`test-smp-boot.cpp`, `smp_install_boot_threads()`), exactly as on the ARM boards — one idle thread per core, each parked in `wait_for_interrupt()`.

Secondary CPUs

A test calls `smp::start_secondary_cores()`, which on this board is one line:

```
void start_secondary_cores () { host_cpu::start_secondary_cpus (); }
```

and in the port:

```

void start_secondary_cpus (void)
{
    for (unsigned cpu = 1; cpu < OS_NCPU; ++cpu)
        pthread_create (&g_cpu_thread[cpu], &attr, secondary_cpu_body, (void*) cpu);

    for (unsigned cpu = 1; cpu < OS_NCPU; ++cpu)    // wait for each to be real
        while (!g_cpu_up[cpu]) ::usleep (100);
}

```

Each secondary then does, in order, exactly what a core does after a spin table releases it:

```

void* secondary_cpu_body (void* arg)
{
    _this_cpu = cpu;                                // 1. know which core I am
    host_cpu::install_fault_stack ();                // 2. be able to report a fault
    ::pthread_sigmask (SIG_BLOCK, &irq_set, nullptr); // 3. interrupts off
    ... fake_thread[cpu] ...                         // 4. somewhere to save
    g_cpu_up[cpu] = true;                            // 5. tell CPU 0
    host_cpu::start_this_cpu_tick ();                // 6. arm my own timer
    g_core_stage[cpu] = 3;                           // 7. the board's "joined" flag
    scheduler::reschedule ();                        // 8. enter the scheduler
    ...never returns...
}

```

The tests wait on `g_core_stage[]` before they start timing anything — which is why every log begins `join: c1=3 c2=3 c3=3 (0 ms)`.

The full sequence, once

```
main()
  scheduler::initialize()
  port::scheduler::initialize()
    build_irq_set
  clear lock_state[], _port_ctx_pending[], _in_isr[]
  host_cpu::install_handlers()    — sigaction(tick), sigaction(IPI),
                                g_cpu_thread[0], sigaltstack
  os_startup_initialize_hardware() — uart.init(), banner,
                                os_startup_initialize_free_store(),
                                exception::init()

  pthread_sigmask(BLOCK)
  new thread "main"              — port::context::create()
  os_startup_create_thread_idle() — port::context::create()
  scheduler::start()
  port::scheduler::start()
    fake_thread[0] becomes current
    os_idle_thread_core[0] = os_idle_thread
    host_cpu::start_this_cpu_tick() — timer_create + timer_settime
    reschedule() → switch_stacks() — first swapcontext: into "main"
                                ↓
os_main()                      (running as a µOS++ thread)
  smp::start_secondary_cores() — pthread_create × (OS_NCPU-1)
  smp_install_boot_threads()   — idle1..idle3
  ... the test ...
```

5. Threads

What a µOS++ thread is here

A `ucontext_t` plus a stack the kernel allocated, both living inside the kernel's `thread` object. There is no host thread per µOS++ thread — that alternative was considered and rejected (§21).

port::context::create()

```
void context::create (void* context, void* func, void* args)
{
    thread::context* th_ctx = (thread::context*) context;
    memset (&th_ctx->port_, 0, sizeof (th_ctx->port_));

    os_impl_ucontext_t* ctx = &th_ctx->port_.ucontext;
    os_impl_getcontext (ctx);                // makecontext requires one

    ctx->uc_link      = nullptr;              // returning is a bug, not an exit
    ctx->uc_stack.ss_sp = th_ctx->stack ().bottom ();
    ctx->uc_stack.ss_size = th_ctx->stack ().size ();
    ctx->uc_stack.ss_flags = 0;

    ::sigemptyset (&ctx->uc_sigmask);        // ← see below
    ::sigaddset (&ctx->uc_sigmask, clock::signal_number ());
    ::sigaddset (&ctx->uc_sigmask, clock::ipi_signal_number ());

    host_cpu::make_entry (ctx, func, args);    // makecontext(trampoline, 2, ...)

    th_ctx->port_.stack_ptr = (stack::element_t*) &th_ctx->port_.ucontext;
}
```

The starting signal mask is the subtle part, and two wrong answers were tried before this one.

- *Inherit the caller's mask* (what `getcontext()` gives you): wrong. A thread is very often created from inside a critical section, so the new thread would start with interrupts masked **for ever** — nothing would ever unmask them.
- *Clear the mask entirely*: wrong the other way, and this one cost a long afternoon. A context resumed by `switch_stacks()` comes back masked and unmask only *after* it has discharged this CPU's deferred publish (§7); a context that has never run arrives on a CPU owing exactly the same publish. Starting it unmasked lets a tick land inside the trampoline **before** that publish happens — and that tick's own switch overwrites the pending slot, so the thread the publish was owed to is never republished and is lost from the ready list for good. The symptom is a thread that simply stops existing.
- *Start masked, like a resumed context*: correct. The trampoline publishes, then unmask, and from that point the thread is preemptible like any other.

The last line publishes `stack_ptr`: a context that has never run is complete by definition, so any CPU may claim it.

The trampoline

`makecontext()` cannot call the kernel's thread body directly, because something has to run *before* the body on a brand-new context:

```
[[noreturn]] void trampoline (void* func, void* args)
{
    host_cpu::publish_pending (); // 1
    ::pthread_sigmask (SIG_UNBLOCK, &interrupts::irq_set, nullptr); // 2
    reinterpret_cast<body_t> (func) (args); // 3
    std::fprintf (stderr, "\n!!! posix-arch: thread body returned\n");
    std::abort ();
}
```

1. Discharge the deferred publish this CPU owes (§7).
2. Only now become preemptible.
3. The kernel's thread body, which does not return. `uc_link` is null, so if it ever did the process would die silently — hence the explicit abort with a message.

`makecontext()` is declared to take `int` arguments, so carrying two *pointers* through it is a portability question rather than an assumption. It was tested on this host (glibc 2.44) before being relied on: a pointer with a non-zero high half survives intact. Upstream's idiom was kept.

Stacks

thread	stack	source
<code>main</code>	64 KiB	the test's <code>static element_t main_stack[8192]</code>
idle (CPU 0)	64 KiB	<code>OS_INTEGER_RTOS_IDLE_STACK_SIZE_BYTES</code>
idle1..N	<code>TEST_IDLE_STACK_WORDS</code>	<code>test-smp-boot.cpp</code> , one per secondary
workers	32 KiB typical	e.g. <code>smp_test2's wstack[OS_NCPU][4096]</code>
fault reporting	64 KiB per CPU	<code>g_fault_stack[OS_NCPU][64*1024]</code> , <code>sigaltstack</code>

32 KiB is the *minimum*, not a suggestion: a signal frame on x86-64 with FPU and AVX state is several kilobytes, and it lands on the thread's stack, and it can nest once (a fault inside a tick, though that one lands on the altstack). Every carried test's worker stacks were already at or above it.

Termination

The port does not special-case a dying thread: `switch_stacks()` always `swapcontext`s, so the outgoing context is always written. Upstream's single-core `reschedule()` had a `save` flag and used `setcontext` when the outgoing thread was `destroyed`; the SMP ARM ports do not, because their assembly restore path always saves too. The thread object outlives the switch — the kernel keeps it on `terminated_threads_list_` until it is reclaimed — so the write is to live storage. Uniform behaviour with the ARM ports was the deciding argument.

6. The scheduler

6.1 Two halves

half	file	what it decides
portable	<code>micro-os-plus-iii-smp/src/rtos/os-core.cpp</code>	<i>which</i> thread runs next
port	<code>micro-os-plus-iii-posix-arch/src/rtos/os-core.cpp</code>	<i>how</i> to start running it

Both are compiled, at the same relative path, exactly as on every other port. They do not collide: the kernel's is `os::rtos::scheduler::...`, the port's is `os::rtos::port::scheduler::...`.

6.2 The picker

`scheduler::internal_switch_threads()` runs under the kernel lock and, in its SMP branch, walks the ready list looking for the first thread that is allowed on this CPU *and* is safe to claim:

```
if (th != nullptr && is_thread_allowed_on_cpu (th, cpu)
    && (th == old_thread || th->context_.port_.stack_ptr != nullptr))
```

Three conditions, and the port only supplies the third. If nothing is found, the CPU takes its own idle thread, `os_idle_thread_core[cpu]`.

6.3 Two locks, and they are not the same lock

	<code>lock_state[cpu]</code>	<code>_smp_klock</code>
kind	per-CPU flag	one recursive lock, global
meaning	"this CPU has the scheduler locked"	"this CPU is inside kernel state"
set by	<code>scheduler::lock()</code> / <code>unlock()</code>	<code>interrupts::critical_section</code>
effect	<code>reschedule()</code> returns early	other CPUs spin
nests	no	yes, by <code>depth</code>

`port::scheduler::locked(state_t)` moves both together, because on this port "the scheduler is locked" also has to mean "this CPU's interrupts are masked":

```
state_t locked (state_t state)
{
    os_assert_throw (!interrupts::in_handler_mode (), EPERM);
    const unsigned cpu = port_cpu_id ();

    if (state == state::locked)
    {
        ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, nullptr);
        state_t tmp = lock_state[cpu];
        if (tmp != state::locked)
        {
            if (_smp_klock.owner != cpu)
                { _smp_klock_raw_acquire (); _smp_klock.owner = cpu; }
            _smp_klock.depth = _smp_klock.depth + 1;
            lock_state[cpu] = state::locked;
        }
        return tmp;
    }
    ... the mirror image, ending in SIG_UNBLOCK ...
}
```

6.4 The kernel-lock release order

Both `critical_section::exit()` and `switch_stacks()` release the lock in this order and no other:

```
_smp_klock.depth = 0;           // 1
_smp_klock.owner = SMP_NO_OWNER; // 2
_smp_klock_raw_release ();      // 3 the lock word, LAST
```

Reverse 3 and 1–2 and this happens: another CPU wins the lock word immediately, installs *its* owner and depth, and then steps 1–2 wipe them. That CPU's `critical_section::exit()` is guarded by `(owner == cpu && depth > 0)`, so it never clears the lock word again — and every CPU spins for ever. A silent, total freeze with no fault and no output.

This is not hypothetical. On the ARM ports it was the root cause of the `smp_test4` hardware deadlock, and the AArch64 port documents it at length. The POSIX port copies the order *and* the reasoning, because a comment that says "order matters" without saying why gets re-ordered by the next person.

6.5 `reschedule()` — the decision tree

```
void reschedule (void)
{
    const unsigned cpu = port_cpu_id ();

    if (rtos::scheduler::locked ()                // (a)
        || (rtos::interrupts::in_handler_mode ()
            && !rtos::scheduler::preemptive ()))
        return;

    if (_smp_klock.owner == cpu && _smp_klock.depth > 0) // (b)
        { _port_ctx_pending[cpu] = 1; return; }

    if (rtos::interrupts::in_handler_mode ()) // (c)
        { _port_ctx_pending[cpu] = 1; return; }

    switch_stacks (nullptr); // (d)
}
```

(a) The scheduler is locked on this CPU, or we are in a handler and the scheduler is cooperative. Nothing to do; the unlock will re-drive it.

(b) *This CPU still owns the kernel lock.* This is the case that matters and it is easy to get wrong. `message_queue::send()` and friends call `resume_one()` from **inside** an `interrupts::critical_section`. Switching there would carry `owner` and `depth` away on the outgoing thread's context — the incoming thread would find a lock it never took, and the next acquirer would wait for ever. So the switch is deferred to the next tick, which runs once that section has exited.

(c) In a handler, but not holding the lock. Marked pending and taken on the way out of the handler (`irq_epilogue`), where the signal mask is right.

(d) Thread mode, lock free: switch now.

`_port_ctx_pending[cpu]` is the port's "PendSV bit". On Cortex-M it literally is one; here it is a word per CPU, set by anyone who wants a switch and consumed by `irq_epilogue()`.

7. The context switch and the deferred publish

This is the one genuinely hard part of the port, and it is a kernel requirement the design spec does not mention.

7.1 The hazard

The picker skips a ready thread whose `context_.port_.stack_ptr` is null. Null means **"not safe to claim"**: the thread's context is still live in another CPU's registers, and resuming it would resume a half-saved context.

On the ARM ports the assembly restore path publishes the value only once SP has left the outgoing stack, so the window is closed by construction. Here there is no assembly, and worse: **by the time `swapcontext()` returns, this CPU is already executing the incoming thread.** The outgoing thread has no code of its own left to run on this CPU.

7.2 The answer

Two decisions.

`stack_ptr` is a flag, not a register store. It is the first member of the context, ahead of the ucontext. The only question the kernel ever asks of it is whether it is null, so any stable non-null value serves; the port uses the address of the context's own ucontext, which is stable and obviously non-null.

The publish is left for whoever arrives next on this CPU. `switch_stacks()` leaves the address and the value in this CPU's slot, and the first thing every resumed context does is discharge it:

```
struct publish_slot { element_t** addr; element_t* val; };
publish_slot g_publish[OS_NCPU];

void defer_publish (unsigned cpu, element_t** addr, element_t* val)
{ g_publish[cpu].addr = addr; g_publish[cpu].val = val; }

void publish_pending (void)
{
    const unsigned cpu = port_cpu_id ();          // re-read: we may have moved
    element_t** addr = g_publish[cpu].addr;
    if (addr == nullptr) return;
    g_publish[cpu].addr = nullptr;
    __atomic_store_n (addr, g_publish[cpu].val, __ATOMIC_RELEASE);
}
```

Two callers, and only two: right after `swapcontext()` returns, and at the top of the trampoline.

This is AArch64's `_smp_pub_addr` / `_smp_pub_val` pair. Same hazard, same answer, different machine. **No kernel change was needed.**

7.3 `switch_stacks()`, annotated

```

stack::element_t* switch_stacks (stack::element_t* sp)
{
    (void) sp;                                // the ARM signature; nothing to pass here

    /* 1. MASK THIS CPU FIRST. */
    sigset_t saved_mask;
    ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, &saved_mask);

    const unsigned cpu = port_cpu_id ();

    /* 2. Take the kernel lock outright. reschedule() has already returned
       early if this CPU still owns it, so it is not held here. */
    _smp_klock_raw_acquire ();
    _smp_klock.owner = cpu;
    _smp_klock.depth = 1;

    rtos::thread* old_thread = host_cpu::current_thread (cpu);

    stack::element_t** pub_addr = &old_thread->context_.port_.stack_ptr;
    stack::element_t* pub_val = (stack::element_t*) &old_thread->context_.port_.ucontext;

    /* 3. Park the outgoing thread BEFORE the picker can see it. */
    __atomic_store_n (pub_addr, (stack::element_t*) nullptr, __ATOMIC_RELEASE);

    /* 4. The kernel picks. */
    rtos::scheduler::internal_switch_threads ();

    rtos::thread* new_thread = host_cpu::current_thread (cpu);
    if (new_thread == nullptr) { trace::printf (...); ::abort (); }

    /* 5. Same thread? Republish, let go, restore the mask, done. */
    if (new_thread == old_thread)
    {
        __atomic_store_n (pub_addr, pub_val, __ATOMIC_RELEASE);
        _smp_klock.depth = 0; _smp_klock.owner = SMP_NO_OWNER;
        _smp_klock_raw_release ();
        ::pthread_sigmask (SIG_SETMASK, &saved_mask, nullptr);
        return nullptr;
    }

    /* 6. Claim the incoming context: from here no other CPU may take it. */
    os_impl_ucontext_t* new_uc = &new_thread->context_.port_.ucontext;
    __atomic_store_n (&new_thread->context_.port_.stack_ptr,
                     (stack::element_t*) nullptr, __ATOMIC_RELEASE);
}

```

```

/* 7. Owe the outgoing thread's publish to the next arrival. */
host_cpu::defer_publish (cpu, pub_addr, pub_val);

/* 8. Release the kernel lock. ORDER MATTERS — see §6.4. */
_smp_klock.depth = 0;
_smp_klock.owner = SMP_NO_OWNER;
_smp_klock_raw_release ();

/* 9. Go. */
if (os_impl_swapcontext (&old_thread->context_.port_.ucontext, new_uc) != 0)
    { trace::printf (...); ::abort (); }

/* 10. Resumed — and NOT necessarily on the CPU that left. Everything
    below must re-read the CPU index. */
host_cpu::publish_pending ();

/* 11. Unmask last: the switch is not interruptible. */
::pthread_sigmask (SIG_SETMASK, &saved_mask, nullptr);
return nullptr;
}

```

Step 3 looks aggressive — the outgoing thread is unpublished before we even know whether we are leaving it. It is correct because the kernel's own test is `th == old_thread || stack_ptr != nullptr`: **this CPU** may always re-pick the thread it is currently running, because it never left it. Only *other* CPUs are locked out, which is exactly the intent.

Step 1 is the fix for a failure that took three attempts to characterise. On ARM, `switch_stacks()` is atomic with respect to this CPU's interrupts for free: the whole function runs inside the IRQ path, with interrupts already masked by the exception entry. Here `reschedule()` reaches it **directly from thread mode**, where the tick is unmasked — so a timer could land between step 3 and step 9 and re-enter this same function on a half-performed switch. That is a genuinely reentrant context switch. The mask is saved into the outgoing ucontext by `swapcontext()` and restored from the incoming one, so it follows the thread across CPUs, which is exactly what interrupt state should do.

7.4 The handover, as a timeline

Two CPUs, one thread **T** moving from CPU 0 to CPU 1.

CPU 0	CPU 1
<pre> switch_stacks() mask; take klock T.stack_ptr := null — T is now invisible to CPU 1's picker internal_switch_threads() → picks U U.stack_ptr := null — U claimed by CPU 0 g_publish[0] := { &T.stack_ptr, val } release klock swapcontext(&T.uc, &U.uc) † T's registers are now safely in T.uc — running as U — publish_pending() T.stack_ptr := val — T is claimable again, by anyone </pre>	<pre> switch_stacks() take klock picker walks the ready list; T is skipped (stack_ptr == null) picks V instead release klock (next switch) picker may now pick T swapcontext(&V.uc, &T.uc) — T is now running on CPU 1 — </pre>

The invariant the design keeps: a thread's `stack_ptr` is non-null only while no CPU is inside `swapcontext()` for it.

7.5 What the publish is *not*

It is not a store of the stack pointer. The register state lives in the ucontext, which `swapcontext()` writes. `stack_ptr` here carries one bit of information and the port never reads the value back. This is why putting it first in the struct is safe and why a "wrong" value would be harmless as long as it is non-null and stable.

7.6 The invariant was measured, not assumed

Two temporary tripwires were compiled in during bring-up and neither ever fired:

- **Kernel-lock exclusivity** — a CPU entering the locked region asserting that no other CPU was already inside.
- **One CPU per thread** — after the picker returns, walking `current_thread_[]` to assert the chosen thread is not current on any other CPU.

The second one is the property the whole deferred-publish design exists to keep. Both were removed before the port was committed; they are recorded here because "it passed" and "the invariant held" are different claims, and this port can make the second one.

8. Interrupts

8.1 Signals are interrupts

the tick	<code>SIGRTMIN</code> , from this CPU's own <code>timer_create</code> timer
the IPI	<code>SIGRTMIN+1</code> , from <code>pthread_kill</code>
<code>irq_set</code>	the set containing exactly those two
"interrupts disabled"	those two blocked in <i>this host thread's</i> mask
"in handler mode"	<code>_in_isr[cpu]</code> , set by the handler

Real-time signals were chosen over `SIGALRM` / `SIGUSR1` for one reason: RT signals **queue**. A CPU that spent two tick periods masked sees two deliveries when it unmask, not one. Standard signals coalesce, and a coalescing tick loses time exactly when the system is busiest.

8.2 Handler installation

```
struct sigaction sa;                // `sigaction` is also a function name;
std::memset (&sa, 0, sizeof (sa)); // the struct tag is required
sa.sa_flags = SA_SIGINFO | SA_RESTART;
::sigemptyset (&sa.sa_mask);
::sigaddset (&sa.sa_mask, clock::signal_number ());
::sigaddset (&sa.sa_mask, clock::ipi_signal_number ());
sa.sa_sigaction = tick_handler; ::sigaction (clock::signal_number (), &sa, nullptr);
sa.sa_sigaction = ipi_handler;  ::sigaction (clock::ipi_signal_number (), &sa, nullptr);
```

- `SA_RESTART`, because the tests print through `write(2)` a thousand times a second and a preempted `write()` must resume rather than fail with `EINTR`.
- `sa_mask` **contains both**, so a tick cannot be interrupted by an IPI or vice versa — the equivalent of not re-enabling interrupts inside the handler.
- **No** `SA_ONSTACK`, and this is the important one. See below.

8.3 Why the tick has no alternate stack

The handler runs on the **µOS++ thread's own stack**. That means the entire signal frame — the saved registers, the FPU state, the `ucontext` the kernel pushed — is part of what `swapcontext()` saves, and therefore migrates with the thread when another CPU resumes it.

An alternate stack (`sigaltstack`) belongs to the *host thread*, i.e. to the CPU. If the tick ran there and then switched away, the frame would stay behind on a CPU the thread has left, and returning through `sigreturn` later would read another CPU's memory.

This is also why the port defines no `OS_HAS_INTERRUPTS_STACK`, where the ARM ports define one and the tests set it up from `__fiq_stack_top` / `__irq_stack_top`. On this machine there is no interrupt stack, and saying so in the header is what makes the carried tests compile that block out.

The **fault** handler is the opposite case and does use `SA_ONSTACK`: it never migrates (it reports and the process ends), and it must work precisely when the stack it would otherwise use is the thing that broke. Getting those two backwards is easy; the code says so at both sites.

8.4 The handlers

```
void tick_handler (int, siginfo_t*, void*)
{
    const unsigned cpu = port_cpu_id ();
    if (cpu >= OS_NCPU) return;           // a host thread that is not a CPU

    interrupts::_in_isr[cpu] = true;

    if (cpu == 0) os_systick_handler ();  // only CPU 0 advances the clock

    scheduler::_port_ctx_pending[cpu] = 1; // every CPU re-picks

    interrupts::_in_isr[cpu] = false;
    irq_epilogue (cpu);
}

void ipi_handler (int, siginfo_t*, void*)
{
    const unsigned cpu = port_cpu_id ();
    if (cpu >= OS_NCPU) return;
    interrupts::_in_isr[cpu] = true;
    scheduler::_port_ctx_pending[cpu] = 1;
    interrupts::_in_isr[cpu] = false;
    irq_epilogue (cpu);
}
```

Deliberately the same shape as `boards/rpi-zero-2w/src/rtos/port_isr.cpp`: flag handler mode, do the work, mark a switch pending, clear the flag, and take the switch on the way out.

Only CPU 0 calling `os_systick_handler()` is the **BCM2837 arrangement exactly**: all four cores take the 1 ms PPI, one of them advances the kernel clock, the others use theirs to preempt themselves.

8.5 The epilogue

```
void irq_epilogue (unsigned cpu)
{
    if (scheduler::_port_ctx_pending[cpu] == 0) return;           // nothing asked
    if (os::rtos::scheduler::locked ()) return;                 // kernel says no
    if (scheduler::_smp_klock.owner == cpu
        && scheduler::_smp_klock.depth > 0) return;             // we hold the lock

    scheduler::_port_ctx_pending[cpu] = 0;
    scheduler::switch_stacks (nullptr);
}
```

Three refusals, each leaving the request pending so the next tick finds it. The third is the one that keeps the kernel lock from travelling: if the interrupted code was inside a critical section, switching now would carry `owner` / `depth` away on the outgoing context.

8.6 The IPI

```
void send_ipi (unsigned cpu)
{
    if (cpu < OS_NCPU && g_cpu_up[cpu])
        ::pthread_kill (g_cpu_thread[cpu], clock::ipi_signal_number ());
}

extern "C" void port_smp_ipi (unsigned cpu) { host_cpu::send_ipi (cpu); }
```

The kernel calls `port_smp_ipi(cpu)` when it makes a thread ready that another CPU should look at. `pthread_kill` to that CPU's host thread is this port's SGI / mailbox doorbell. The `g_cpu_up[]` guard matters: signalling a `pthread_t` that was never created is undefined, and the secondaries come up asynchronously.

8.7 Async-signal-safety, in practice

The switch happens **inside** the handler, which is not something POSIX blesses in general. It is well defined here for a specific reason: the handler frame lives on the thread's own stack, so resuming that context resumes the handler, which then returns through `sigreturn` normally. Nothing is executed out of order; the frame is merely paused, possibly on another CPU.

What follows from that, and is a rule for anyone touching this port:

in a handler, or reachable from one	verdict
<code>write(2)</code>	fine — <code>uart::uart1</code> and the fault reporter use only this
<code>printf</code> , <code>puts</code> , any stdio	not safe; a test using them may interleave
<code>malloc</code> / <code>new</code>	not safe on the host allocator; <code>μOS++</code> 's own is guarded by the klock
<code>pthread_mutex_*</code>	not safe — this is why the kernel lock is a raw spin
<code>pthread_sigmask</code>	safe
<code>pthread_kill</code>	safe
<code>__atomic_*</code>	safe
<code>abort</code> / <code>_exit</code>	safe
<code>backtrace</code>	not strictly safe; used only on the fatal path, deliberately

`mutex-stress` prints with `printf` because it is upstream's test and is not edited; it is the single-core leg, so nothing interleaves.

9. Preemption

This port is preemptive. That is the headline change from upstream and it is what makes the nine carried SMP tests meaningful rather than decorative.

The life of a tick

```
t      the timer expires for CPU 2
t+ε    the host kernel delivers SIGRTMIN to host thread 2 only
       (SIGEV_THREAD_ID), pushing a signal frame onto the RUNNING
       μOS++ THREAD's stack, and adding SIGRTMIN+SIGRTMIN+1 to the mask
----- tick_handler()
       _in_isr[2] = true
       (cpu 0 only: os_systick_handler() – timers, sysclock)
       _port_ctx_pending[2] = 1
       _in_isr[2] = false
irq_epilogue(2)
       pending? yes. scheduler locked? no. we hold klock? no.
       switch_stacks(nullptr)
       mask (already masked by the handler; now explicitly)
       klock; unpublish old; pick; claim new; defer publish; unlock
       swapcontext -----
----- |
       ...CPU 2 is now running a different μOS++ thread... |
       |
       (later, possibly on a DIFFERENT CPU) |
       |
       ←-----
       publish_pending()
       restore mask
       return from switch_stacks()
       return from irq_epilogue()
       return from tick_handler()
sigreturn — the interrupted instruction resumes
```

The interesting line is the one where the arrow comes back: the code after `swapcontext()` may be executing on a different host thread from the code before it. Everything below that line re-reads the CPU index. Nothing captured above it is valid.

When preemption does not happen

Three deferrals, all in §6.5 and §8.5, all leaving `_port_ctx_pending[cpu]` set: the scheduler is locked on this CPU; this CPU owns the kernel lock; or we are in a handler and the scheduler is cooperative. Each is re-driven by the next tick, 1 ms later.

The cost, stated plainly

A context switch here is: two `pthread_sigmask` syscalls, one atomic exchange (usually uncontended), the kernel's list walk, and a `swapcontext` — which is itself a `rt_sigprocmask` syscall plus a register save/restore. Call it a few microseconds. On a Cortex-A53 the same switch is a few hundred nanoseconds of assembly.

This is not a defect and it is not worth optimising. The purpose of the port is to make interleavings *reachable*, not to be fast. Indeed the slowness is what found the mutex race: see §18.

10. Time

The tick

Each CPU arms its own:

```
sigevent sev; memset (&sev, 0, sizeof (sev));
sev.sigev_notify      = SIGEV_THREAD_ID;
sev._sigev_un._tid     = (int) syscall (SYS_gettid);    // THIS host thread
sev.sigev_signo        = clock::signal_number ();

timer_t timer;
timer_create (CLOCK_MONOTONIC, &sev, &timer);

const long period_ns = 1000000000L / OS_INTEGER_SYSTICK_FREQUENCY_HZ; // 1 ms
itimerspec its = { { 0, period_ns }, { 0, period_ns } };
timer_settime (timer, 0, &its, nullptr);
```

`CLOCK_MONOTONIC`, not `CLOCK_REALTIME`: the tick must not jump when the wall clock is stepped. `SIGEV_THREAD_ID` is Linux-specific; the `#else` branch falls back to `SIGEV_SIGNAL`, which is process-directed and therefore only correct at `OS_NCPU=1` — the file says so.

`port::clock_systick::start()` is an empty function with a comment, because the arming happens at CPU start rather than at kernel-clock start:

```
void clock_systick::start (void)
{
    // Each CPU arms its own timer when it starts; this is CPU 0's, and the
    // secondaries' are armed by host_cpu::start_secondary_cpus(). Only CPU 0
    // advances the kernel clock -- exactly as on the BCM2837, where all four
    // cores take a 1 ms PPI but only core 0 calls os_systick_handler().
}
```

The high-resolution clock

```
uint32_t clock_highres::input_clock_frequency_hz (void) { return 1000000; }
```

`CLOCK_MONOTONIC` read at microsecond resolution. Claiming nanoseconds would be claiming a precision the read does not have once the syscall (or vDSO call) and the division are counted. `cycles_per_tick()` and `cycles_since_tick()` are deltas against a timestamp taken at the last tick.

The **board's** free-running counter, `timer_arm::get_count()`, is a separate thing and does report nanoseconds: it is the stand-in for `CNTPCT`, and `smp-mat-test` times its passes with it.

What a "1 ms tick" means on a host

It means "the host kernel will try". Under load, or with `OS_NCPU` above the number of real cores, deliveries drift and queue. Because the signal is real-time, they are not lost — they arrive late and in a burst. Tests that measure elapsed *kernel* time (`sysclock.now()`) stay consistent; tests that compare kernel time against wall time will see skew. None of the carried tests does the latter.

11. Memory

The free store

The kernel's `src/startup/initialise-free-store.cpp` is `__ARM_EABI__`-guarded from top to bottom, so on a host it compiles to nothing and `os_startup_initialize_free_store()` is undefined — which is exactly what every carried test calls. Relaxing that guard would mean editing the kernel, which is kept merge-clean against upstream. So the port supplies the hook, which is what a port is for.

```
using application_memory_resource = os::memory::first_fit_top;

alignas (application_memory_resource)
char application_free_store[sizeof (application_memory_resource)];

void __attribute__ ((weak))
os_startup_initialize_free_store (void* heap_address, std::size_t heap_size_bytes)
{
    new (&application_free_store)
        application_memory_resource { "app-heap", heap_address, heap_size_bytes };
    ...->out_of_memory_handler (os_rtos_application_out_of_memory_hook);
    estd::pmr::set_default_resource (...);
}
```

Not `malloc`, and the reason is the subject of the whole port. Upstream's posix-arch let the system `malloc` serve, and its `NOTES.md` says so; that was correct for upstream, which had one host thread. Here a μ OS++ thread may be preempted inside an allocation and resumed on a *different* host thread, and glibc's malloc is thread-safe by taking an arena lock — which would then be released by a host thread that did not take it.

Using `µOS++`'s own resource removes the question instead of betting on it: the allocator is the kernel's, and its mutual exclusion is the kernel's scheduler lock, which already migrates correctly because making it do so is what the rest of this port is about.

There is a second benefit, and it is the reason to prefer this even if `malloc` were safe: **a `first_fit_top` over a fixed block is what every board does**. An allocation pattern that exhausts the heap on a Pi exhausts it here too, instead of being quietly absorbed by a host with gigabytes to spare. The port behaves like the machines it is standing in for.

No `sbrk()` adjustment is made. On a bare-metal target the kernel's version pushes `sbrk` past the free store so newlib's `malloc` cannot collide with it; here the block is ordinary static storage the host already owns.

The heap block, and one piece of assembly

The carried tests compute their heap size the way bare-metal code does:

```
os_startup_initialize_free_store (__heap_start,
                                (std::size_t) (__heap_end - __heap_start));
```

`__heap_start` and `__heap_end` must therefore be **labels**, not objects. A first attempt defined `__heap_end` as a `char* const`, which would have made the subtraction the distance between two pointers rather than the size of the block. The board defines them with ELF inline assembly — the only assembly in the whole port, and it is about the object format rather than the CPU:

```
__asm__ (".section .bss.uos_heap,\"aw\",@nobits\n"
        ".balign 16\n"
        ".globl __heap_start\n.hidden __heap_start\n__heap_start:\n"
        ".zero " UOS_STR (UOS_BOARD_HEAP_BYTES) "\n" /* 32 MiB */
        ".globl __heap_end\n.hidden __heap_end\n__heap_end:\n"
        ".size __heap_start, __heap_end - __heap_start\n"
        ".previous\n");
```

`@nobits` keeps 32 MiB out of the executable; `.hidden` keeps the symbols from being interposed. Every log line `first_fit_top(0x...,33554432) @0x... app-heap` is this block being handed over.

Stacks

Covered in §5. The one number to remember: **32 KiB minimum**, because a signal frame lands on the thread's stack.

12. Faults and diagnostics

On this machine a synchronous fault is a signal. `exception::init()` installs handlers for `SIGSEGV`, `SIGBUS`, `SIGFPE` and `SIGILL` with `SA_SIGINFO | SA_ONSTACK`, and each CPU has its own 64 KiB alternate stack so a blown stack can still be reported.

Everything in the reporting path uses `write(2)` and nothing else: no `printf`, no `malloc`, no locks. A reporting path that can itself deadlock reports nothing — which is precisely the state this port was in before `os_startup_initialize_hardware()` was wired up (§4).

What a fault prints

```
!!! FATAL EXCEPTION on CPU 1 -- signal Segmentation fault at 0x00000000000000C2,
                                pc 0x000055AF745AA8C6 (static 0x0000000000001A8C6)

cpu0: idle
cpu1: w1
cpu2: idle2
cpu3: w3
klock: owner=0x0000000000000003 depth=0x0000000000000000 lock=0x0000000000000000
backtrace (static):
    0x0000000000001E37F
    0x0000000000001E608
    0x000029AFC44AE6F0    ← the signal trampoline
    0x0000000000001A8C6    ← the faulting frame
    0x000000000000172DD
    0x00000000000017695
    ...
```

Line by line:

- **at** — `si_addr`, the address that could not be accessed. `0xC2` is a member offset from a null `this`, which is a diagnosis in itself.
- **pc** — the faulting instruction, taken from the `ucontext` the kernel handed the handler (`gregs[REG_RIP]` on x86-64, `uc_mcontext.pc` on AArch64, and so on). The ARM ports report `ELR` for the same reason.
- **(static ...)** — `pc` minus the image's load bias, so it can be looked up directly in the ELF file. The bias is captured with `dladdr()` in `exception::init()`, i.e. in a sane context, because `dladdr` is not async-signal-safe.
- **cpuN:** — which `µOS++` thread each CPU was running. On a machine with several CPUs, "it crashed" says nothing without this.
- **klock:** — owner, depth and lock word. Note that a snapshot taken by *another* CPU mid-release can legitimately look inconsistent (`owner=3 depth=0 lock=0`); that is the release sequence of §6.4 caught in the act, not corruption.

- `backtrace` — return addresses, already bias-corrected.

Then the handler restores `SIG_DFL` and re-raises, so the shell sees the real signal and a core file is still produced.

Turning that into a line number

```
addr2line -f -C -i -e test/build/test/smp_test2-host 0x1a8c6 0x172dd
```

`-i` matters: almost everything is inlined at `-O2`, and without it you get the outermost function and a useless `???:?`.

This machinery is not debugging scaffolding left behind — it is the port's fault reporter and it is how §18 was solved after three speculative fixes had resolved nothing. It costs one `dladdr()` at startup.

13. The board

`test/boards/native/` is thin, and honestly so. On the BCM2837 the equivalent files own the spin table, the mailbox doorbell and the cache maintenance around them, because releasing a core is a silicon act. Here the silicon *is* the host kernel, so the act belongs to the port (`host_cpu`) and the board only names it — the same division of labour, applied honestly rather than copied.

`board.cmake`

```
set (UOS_BOARD_SRC_DIR "${CMAKE_CURRENT_LIST_DIR}")
set (UOS_BOARD_FLAGS "") # no -mcpu: the host targets the host

set (NCPU 4 CACHE STRING "Host threads acting as CPUs")
set (UOS_BOARD_NCPU ${NCPU})

set (UOS_BOARD_CAPS smp sdcard led) # NOT usb-device
set (UOS_BOARD_LIBS "")
set (UOS_BOARD_DEVICES micro-os-plus::devices-hostfile)
set (UOS_BOARD_DEFINES "PORT_GREETING=\"µOS++ POSIX synthetic host (host threads as CPUs)\")
```

Two board facts the silicon boards have and this one has not, and the absence *is* the statement:

- `UOS_BOARD_LINKER_HW` — the host toolchain links with its own script.
- `UOS_BOARD_LINKER_QEMU` — the host **is** the machine; there is nothing to emulate. A test therefore builds **one** image, the way the hardware-only Luckfox Lyra does, for the mirror-image reason.

`NCPU` is the one board fact on this machine that can honestly be changed from the command line, because the "silicon" is a thread count.

`sdcard` is claimed because the host-file back-end *is* a block device the SD tests can reach — not because the machine has a slot. `usb-device` is not claimed: nothing here can be a USB gadget, and claiming it would make `usb_test` appear and fail.

The four headers

They exist so the carried tests compile **unchanged**. A test names `uart::uart1` and must not have to know whether that is a PL011 or a pipe.

header	what it is here	note
<code>uart.hpp</code>	fd 1, via <code>write(2)</code>	async-signal-safe and unbuffered, so a deadlocked test still shows everything it printed
<code>led.hpp</code>	<code>[LED on]</code> / <code>[LED off]</code>	printed only on change , so a 1 Hz blink is two lines a second
<code>timer_arm.hpp</code>	<code>CLOCK_MONOTONIC</code> at 1 GHz	the stand-in for <code>CNTPCT</code> ; <code>init()</code> / <code>start_1ms()</code> are no-ops because the port owns the tick
<code>smp.hpp</code>	<code>start_secondary_cores()</code> , <code>g_core_stage[]</code>	same names, same stage numbers as the ARM boards

`led.hpp` prints rather than pretending, and the comment says why: this project has already been bitten once by a blinking test driving GPIO16 on a board whose LED is GPIO29 — nothing lit, and nobody noticed. Output a runner can read cannot fail that way.

`uart.hpp` also supplies `PORT_BANNER_ISA`, so a carried test's banner reads `(x86-64)` instead of naming an ARM part.

The SD card is a file

Two of the carried tests reach a block device. `micro-os-plus-iii-devices/include/sd.hpp` dispatches on a define and its comment states the contract: a new back-end arrives as a **new file** and a **new target**, defining `sd::SdCard` with the same eight methods and the same failure vocabulary.

```
(default)          Arasan SDHCI @ 0x3F300000 (BCM2837) -- sd.cpp
SD_BACKEND_DWMMC    Synopsys DW-MMC @ 0xFF480000 (RK3506) -- soc/rk3506/
SD_BACKEND_HOSTFILE an image file on the host (POSIX)  -- soc/native/
```

`soc/native/src/sd_hostfile.cpp` is `pread` / `pwrite` on a file: 512-byte sectors, a sector count read once at `init()`, range-checked transfers. Two environment variables:

variable	default	
<code>UOS_SD_IMAGE</code>	<code>disk.img</code>	the backing file; created if missing
<code>UOS_SD_SECTORS</code>	<code>65536</code>	32 MiB; an existing image grows, never shrinks

FatFs is deliberately **absent** from the `devices-hostfile` target. The two tests take their `!HW_BUILD` branch, which formats a flatfs volume; the FatFs branch mounts a board's real boot partition, which a host file is not. Linking `ff.c` anyway would be weight nothing exercises.

The image is never deleted, so a run can be examined afterwards with any ordinary tool — a genuine advantage of this back-end over the two real ones, not a compromise.

The BCM2837 and RK3506 back-ends are byte-for-byte unchanged, which is the whole point: `smp-num-test` and `smp-pipeline-test` run natively with **no edit to either test**.

The board's two sources

- `src/heap.cpp` — the 32 MiB heap block and its two labels (§11).
- `src/smp.cpp` — `g_core_stage[OS_NCPU]` and the one-line `start_secondary_cores()`.

14. The build

Layout and dependency resolution

```
get_filename_component (_uos_siblings "${CMAKE_CURRENT_SOURCE_DIR}/.." ABSOLUTE)
set (UOS_SMP_DIR      "${_uos_siblings}/micro-os-plus-iii-smp"    CACHE PATH ...)
set (UOS_DEVICES_DIR "${_uos_siblings}/micro-os-plus-iii-devices" CACHE PATH ...)
```

Clone the three repositories side by side and it works; override either with `-D` to build against a working copy elsewhere. A missing dependency produces a `FATAL_ERROR` that tells you the clone command, not a stack of CMake noise.

Boards are discovered, not listed

```
file (GLOB _board_decls CONFIGURE_DEPENDS "test/boards/*/board.cmake")
```

Every directory under `test/boards/` holding a `board.cmake` is a board, and that file is the only place its facts are written. Nothing in the port's `CMakeLists.txt` knows any board's name; `-DBOARD=` with an unknown name lists the ones that exist.

The port then *requires* four variables (`UOS_BOARD_SRC_DIR`, `UOS_BOARD_NCPU`, `UOS_BOARD_CAPS`, `UOS_BOARD_DEFINES`) and checks that a board declaring the `sdcard` or `usb-device` capability also names a `UOS_BOARD_DEVICES` to provide it. `UOS_BOARD_LINKER_HW` is deliberately **not** required.

The port target

```
add_library (micro-os-plus-iii-posix-arch-interface INTERFACE)
target_sources (... INTERFACE
    src/rtos/os-core.cpp src/host_cpu.cpp src/exception_handler.cpp
    src/free-store.cpp src/diag/trace-posix.cpp src/board-contract.cpp
    ${_board_sources})
target_compile_definitions (... INTERFACE _XOPEN_SOURCE=700 _GNU_SOURCE ${UOS_BOARD_DEFINES})
target_link_libraries (... INTERFACE micro-os-plus::iii pthread rt ${UOS_BOARD_LIBS})
add_library (micro-os-plus::posix-arch ALIAS ...)
```

An INTERFACE library, so every application compiles its own copy of the port with its own `OS_NCPU` — which is what makes per-test CPU counts possible.

The test loop

`test/CMakeLists.txt` is the same loop as the three cross-compiled projects, with one difference and it is a difference in the machine rather than in the method: **one variant**, `host`, because a test *is* an executable.

It is deliberately **not** a `HW_BUILD`. That define means "use the board's real FAT32 boot partition"; the SD tests' other branch is flatfs over a plain image file, which is exactly what a host-file back-end is. Claiming `HW_BUILD` here would silently point the SD tests at a partition that does not exist.

The same six per-test hooks as the other ports, with the same defaults and the same meanings, so a test can be carried between ports without its build rules changing meaning:

hook	what it answers
<code>board_test_defines(app)</code>	extra <code>-D</code> for this test
<code>board_test_sources(app)</code>	extra sources
<code>board_test_includes(app)</code>	extra include directories
<code>board_test_linker(app)</code>	(no-op here — a host executable has no script)
<code>board_test_options(app)</code>	compile options on the test's own sources
<code>board_test_ncpu(app)</code>	how many CPUs one test runs on
<code>BOARD_TEST_NEED_DEVICES</code>	link <code>UOS_BOARD_DEVICES</code>
<code>BOARD_TEST_SELF_CONTAINED</code>	do not add <code>test/<board>/src/*.cpp</code>
<code>BOARD_TEST_NO_KERNEL</code>	(unused here)

`test/native/tests.cmake` uses three of them:

```
set (BOARD_TEST_NEED_DEVICES smp-num-test smp-pipeline-test)

function (board_test_ncpu _app _out)
  if (_app STREQUAL "mutex-stress")
    set (${_out} 1 PARENT_SCOPE)
  else ()
    set (${_out} "${UOS_BOARD_NCPU}" PARENT_SCOPE)
  endif ()
endfunction ()

set (BOARD_TEST_SELF_CONTAINED mutex-stress)
```

`uos_add_app()` turns `NCPU` into defines:

```
target_compile_definitions (${_name} PRIVATE OS_NCPU=${A_NCPU})
if (A_NCPU GREATER 1)
  target_compile_definitions (${_name} PRIVATE OS_USE_SMP_SCHEDULER=1)
endif ()
```

so one build tree holds nine four-CPU tests and one single-CPU test, each with its own copy of the kernel and the port.

Commands

```
cd micro-os-plus-iii-posix-arch
cmake -S . -B test/build -G Ninja -DBOARD=native -DCMAKE_BUILD_TYPE=Release
cmake --build test/build -j8
```

No toolchain file: the host compiler targets the host. `-DNCPU=<n>` changes the board's CPU count.

15. The tests

Ten applications. Nine are the SMP object tests carried from the ARM boards — **not rewritten, not adapted**: the same `main.cpp`, compiled against the same `uart.hpp` / `led.hpp` / `smp.hpp` / `timer_arm.hpp` API. The tenth is upstream's own.

test	CPUs	what it exercises	runtime
<code>smp_test0</code>	4	console + 1 ms tick on core 0; ten <code>sleep_for(1000)</code> heartbeats	~10 s
<code>smp_test1</code>	4	semaphore ping-pong across cores + LED	~10 s
<code>smp_test2</code>	4	mutex coherency : 4 workers × 2000 increments under one mutex	~2 s
<code>smp_test3</code>	4	<code>message_queue</code> producer/consumer, 500+ messages	~12 s
<code>smp_test4</code>	4	no-affinity load balancing : 8 workers, per-CPU tallies	~10 s
<code>smp-mat-test</code>	4	parallel block linear solver, N=120 B=20, vs. a classical solver	~2 s
<code>smp-pro-cons-test</code>	4	every kernel object : 12 threads, pool, queue, semaphores, mutex, condvar, event flags, timer, sysclock, yield/suspend/resume	~5 s
<code>smp-num-test</code>	4	UART + LED + SD + FPU; writes <code>num.txt</code> to a flatfs volume	~60 s
<code>smp-pipeline-test</code>	4	13 threads, shared queues, SD persistence , <code>yield()</code> stress	~90 s
<code>mutex-stress</code>	1	10 threads on one mutex; fairness/uniformity statistics	~15 s

What each one actually proves *here*

`smp_test0` — the port boots, the console works, CPU 0's timer fires at 1 kHz and `sysclock` advances. The minimum viable port.

```
[tick] heartbeat 0  os_clock=0 ms
...
[tick] heartbeat 9  os_clock=9000 ms
RESULT: PASS (10 heartbeats on core 0)
```

`smp_test1` — secondary CPUs come up (`join: c1=3 c2=3 c3=3`), a semaphore wakes a thread on another CPU, and **threads migrate**: the reporter thread is on c2 while ping is on c0 and pong on c1, and that assignment was not configured anywhere.

```

join: c1=3 c2=3 c3=3 (0ms)
[c0] rounds=2    ping@c0 pong@c0  t=0ms
[c2] rounds=1002 ping@c0 pong@c1  t=1000ms
...
RESULT: PASS

```

smp_test2 — four CPUs, one μ OS++ mutex, 8000 increments that must not lose one. This is the test that found the kernel race (§18) and the one to run in a loop when touching anything in §6 or §7.

```

[c3] worker done, counter=6584
[c0] worker done, counter=7016
[c2] worker done, counter=7237
[c1] worker done, counter=8000
counter = 8000  expected = 8000
PASS: lock coherent across 4 cores

```

smp_test3 — **message_queue** across CPUs, which is the path where **resume_one()** is called from inside a critical section. It is therefore the test that exercises **reschedule()**'s deferral case (b) hardest.

smp_test4 — the load-balancing test, and the clearest evidence that migration works. Eight workers, per-CPU tallies, reported each second:

```

-- reporter on c0 t=9007ms --
w0 [ c0=3297 c1=3794 c2=3810 c3=3683 ]
w1 [ c0=3278 c1=3646 c2=3927 c3=3706 ]
w2 [ c0=3231 c1=3783 c2=3796 c3=3716 ]
...

```

Every worker has run on every CPU, thousands of times, in roughly equal measure. That is **stack_ptr** and the deferred publish working, ten thousand times a second, for nine seconds.

smp-mat-test — a real numerical workload with a correctness check: a parallel block solver against a classical one, compared element by element.

```

Total Absolute Difference (Sum): 0.000000
Max Absolute Difference:         0.000000
  Parallel Block Solver: 1058.000 us
  Classical Solver:      546.000 us
  Speedup Ratio:         0.516x
RESULT: PASS (block == classical within tolerance)

```

The speedup is **below 1**, and that is expected: four "CPUs" that are host threads on an oversubscribed machine, with a switch costing microseconds, do not beat a single-threaded solver on a 120×120 problem. The test asserts *correctness*, not speed, and correctness is what it reports.

smp-pro-cons-test — the broadest test in the suite: every kernel object at once, for four seconds, with a summary that counts each one.

```
[2] memory_pool      : allocs=1069, frees=1069 (diff=0)
[3] message_queue    : sent=1069, received=1069
[5] semaphore_binary : 134 stage synchronizations verified
[7] condition_variable : 66 batches signaled & verified
[9] timer            : 22 software timer ticks delivered
[11] yield/suspend/res : 2138 yields, 11 suspends, 11 resumes
Produced by core : c0=404 c1=131 c2=285 c3=249
Consumed by core : c0=276 c1=117 c2=335 c3=341
RESULT: PASS
```

This one needed the single edit made to any carried test on this board: its `hw_result::ok()` call is wrapped in `#if defined(SEMIHOST)` on the ARM copies, because there the call compiles to a semihosted `SYS_EXIT` and a plain image must fall through to an idle blink. On the host there is no "without it" — a test that idled for ever would have to be killed by the runner and could report nothing — so the native copy calls it unconditionally, like the other eight.

smp-num-test and **smp-pipeline-test** — the SD pair. They format a flatfs volume on `sd:SdCard`, write to it from several CPUs and read it back:

```
Initialising SD card via SDHCI @0x3F300000...
SD card ready: 65536 sectors (~32 MiB)
flatfs volume formatted.
```

(The "SDHCI @0x3F300000" is the carried test's own text; the back-end behind it is a file.) These two are the reason **micro-os-plus-iii-devices** gained a third back-end, and they run with **no edit at all**.

Their per-core distribution is lopsided — `Produced by Core: c0=3275 c1=0 c2=0 c3=0` in the pipeline test — and that is honest rather than broken: those producers block on I/O rather than on CPU, so the thread rarely reaches a preemption point and rarely moves. The same test on silicon shows a similar shape.

mutex-stress — §17.

What is not carried

Three tests that need real hardware and would test the host rather than the kernel:

test	why not
<code>sd_test</code>	wants a seeded FAT32 boot partition , not a formattable image
<code>smp-mat-sdcard-test</code>	the same card, as a 4 GiB working set
<code>usb_test</code>	USB device mode; the board does not claim <code>usb-device</code>

16. Running them

```
BOARD=native test/run.sh          # the whole suite
BOARD=native test/run.sh smp_test2 # one test, echoed to this terminal too
```

`test/run.sh` is a dispatcher and nothing else — the sibling of the ARM ports' `test/qemu.sh`. It contains no board names: it hands off to `test/boards/<id>/run.sh`, so adding a board adds a directory. An unknown name gets the available boards listed rather than a wrong branch taken.

The board's runner is three lines of facts over the shared runner:

```
UOS_RUN_ONLY="${1:-}" exec "$SMP_DIR/test_smp/run-host.sh" "$BUILD/test"
```

`test_smp/run-host.sh` is `run-qemu.sh` with the emulator taken out: the same timeout table, the same log directory, the same PASS / SKIP / FAIL / TIMEOUT / NO-RESULT verdicts read from the `RESULT:` line each test prints, the same summary line and the same exit status. It gives each SD test its own image under the log directory so runs do not share a volume.

env	
<code>BOARD</code>	which board (default <code>native</code>)
<code>BUILD</code>	the CMake build directory (default <code>test/build</code>)
<code>UOS_RUN_ONLY</code>	one test; set by passing an argument
<code>UOS_SMP_DIR</code>	where the kernel repo is, if not a sibling

Logs land in `<build>/test/.host-logs/<app>.log`, and SD images beside them as `<app>.disk.img`.

```

mutex-stress          PASS
smp-mat-test          PASS
smp-num-test          PASS
smp-pipeline-test     PASS
smp-pro-cons-test     PASS
smp_test0             PASS
smp_test1             PASS
smp_test2             PASS
smp_test3             PASS
smp_test4             PASS

host suite: 10 passed, 0 skipped, 0 failed

```

The verdict is the exit status

`hw_result::ok()` / `fail()` on the silicon ports compile to a semihosted `SYS_EXIT` under `SEMIHOST` and to nothing otherwise, so a plain image keeps its idle-forever behaviour. Here there is no "otherwise": the host always has an exit status, and a test process that idled for ever would have to be killed by the runner and could report nothing. So they always end the run.

```

[[noreturn]] inline void ok () noexcept
{
    std::fflush (nullptr);          // ← see below
    ::_exit (0);
}

```

`_exit()`, not `exit()` — other CPUs are still running threads, and `exit()` would run static destructors underneath them, turning a clean PASS into a crash in the teardown.

...but `_exit()` **does not flush stdio either**. Most tests here print through `uart::uart1`, which is `write(2)` and therefore already on its way out; a test carried from upstream's suite prints with `printf()`, and to a pipe — which is how the runner captures it — stdout is fully buffered. Without the flush its entire output, verdict included, is discarded at the exit. Found exactly that way: `mutex-stress` ran, passed, and printed nothing at all.

17. `OS_NCPU=1`

The port is dual-branch on `OS_USE_SMP_SCHEDULER`, which `uos_add_app()` derives from `NCPU GREATER 1`. This is the same arrangement `cortexm` uses to run its three STM32 boards at `OS_NCPU=1` off the RP2350's SMP core, and it is the first clause of step 5's gate.

The surface is small, because the kernel spells only one thing two ways:

```
#if defined(OS_USE_SMP_SCHEDULER)
    extern thread* volatile current_thread_[OS_NCPU];
#else
    extern thread* volatile current_thread_;
#endif
```

So the port has one accessor, in `host_cpu.hpp`, and nothing else in the port mentions the difference:

```
inline os::rtos::thread* volatile& current_thread (unsigned cpu)
{
    #if defined(OS_USE_SMP_SCHEDULER)
        return os::rtos::scheduler::current_thread_[cpu];
    #else
        (void) cpu; return os::rtos::scheduler::current_thread_;
    #endif
}
```

`os_idle_thread_core[]` is the only other SMP-only kernel symbol the port names, and its two uses are guarded. Everything else — the klock, `lock_state[]`, `_in_isr[]`, `_port_ctx_pending[]`, the deferred publish — stays exactly as it is, as arrays of one. It costs nothing and it means there is one code path to reason about rather than two.

What runs there

The nine SMP object tests **cannot** be built single-core by construction: they call `thread::cpu_affinity()`, which does not exist without the SMP scheduler. So the single-core leg is `mutex-stress`, carried from the kernel's own `tests/sources/mutex-stress` — upstream's test, not one invented for the purpose, and the self-contained one of the two candidates (`rtos-apis` pulls in FatFs and posix-io).

It is wired through `board_test_ncpu`, and it is also `BOARD_TEST_SELF_CONTAINED`, because the board's shared test support is the SMP boot helper: `test-smp-boot.cpp` installs per-core idle threads through `scheduler::os_idle_thread_core[]`, which exists only under the SMP scheduler.

Two changes were made to the carried copy, and no third: `RUN_SECONDS` defaults to 10 rather than 30 (the runner passes no argv, and upstream's 30 s was chosen for a person watching the distribution converge), and the `RESULT:` line plus `hw_result` at the end, which is the verdict convention every test in this workspace follows.

It is apt as well as available: ten threads hammering one mutex is the shape of the defect this port found the same week.

```

µOS++ IIIE version 7.0.1
POSIX synthetic SMP, running on x86_64 Linux 7.2.6-arch2-1; 1 CPU, 1000 Hz tick, preemptive
...
[ 5s] t0:36 t1:40 t2:38 t3:36 t4:37 t5:41 t6:40 t7:40 t8:39 t9:39
      sum=386, avg=39, sigma=1, delta in [-3,2] [-7%,5%]
[ 10s] t0:74 t1:76 t2:80 t3:72 t4:76 t5:77 t6:81 t7:81 t8:84 t9:82
      sum=783, avg=78, sigma=3, delta in [-6,6] [-7%,8%]
[ 15s] t0:107 t1:116 t2:125 t3:112 t4:115 t5:115 t6:123 t7:121 t8:126 t9:117
      sum=1177, avg=118, sigma=5, delta in [-11,8] [-8%,7%]
Done.

RESULT: PASS

```

$\sigma = 5$ on a mean of 118 over ten threads: the mutex is not merely correct, it is fair. That is a property the SMP tests do not measure and a single-core test can.

18. The defect this port found

The port's first full multi-core run of `smp_test2` faulted. At `OS_NCPU=4` it failed **21 runs out of 25**, with `this == nullptr` inside `thread::priority_inherited(priority_t)`.

How it was found

Three speculative fixes had already been tried and none resolved it, so the approach changed from hypothesising to measuring.

1. **Make the failure reportable.** The first attempts produced *nothing at all* — no fault report. The cause was §4's missing `os_startup_initialize_hardware()`: `exception::init()` had never run. Wiring the startup hooks into `port::scheduler::initialize()` turned a silent death into a report.
2. **Make the report useful.** The handler was taught to read the faulting `pc` out of the signal's `ucontext`, to subtract the image's load bias (captured with `dladdr()` at init, not in the handler), and to print a `backtrace()`.
3. **Read it.** `at 0x...C2, pc 0x...8C6 (static 0x1A8C6) backtrace (static): ... 0x1A8C6 0x172DD 0x17695 ...`
`sh $ addr2line -f -C -i -e smp_test2-host 0x1a8c6 0x172dd 0x17695`
`os::rtos::thread::priority_inherited(unsigned char)`
`os::rtos::mutex::internal_try_lock_(os::rtos::thread*) os::rtos::mutex::lock()`
4. **Confirm at the instruction.** `objdump` showed `cmp %sil,0xc2(%rbx)` with `%rbx = 0` — `this` null, `0xC2` the offset of `prio_inherited_`. Not a corrupted pointer: a null one.

Two invariant tripwires (kernel-lock exclusivity, one-CPU-per-thread) were compiled in during the same investigation and never fired, which ruled out the port's own machinery before the kernel was looked at.

The defect

`micro-os-plus-iii-smp/src/rtos/os-mutex.cpp`, the priority-inheritance path of `mutex::internal_try_lock()`:

```
// Boost owner priority.
if ((boosted_prio_ > owner_>priority_inherited ()))
{
    // ----- Enter uncritical section -----
    scheduler::uncritical_section sucs;

    owner_>priority_inherited (boosted_prio_);    // ← null deref
    // ----- Exit uncritical section -----
}
```

`scheduler::uncritical_section` **releases** the kernel lock. It must: `priority_inherited()` ends in `this_thread::yield()`, which cannot run scheduler-locked. But while it is open, the mutex's owner — running on another CPU — can complete its own `unlock()`, and `internal_unlock_()` ends with:

```
owner_ = nullptr;
count_ = 0;
```

The compiler must re-read the member after the opaque call, so the second line inside the uncritical section dereferences a null pointer.

Every SMP port has this window. It is rare on hardware because `scheduler::unlock()` / `lock()` is a handful of instructions. On the POSIX host it is *two* `pthread_sigmask()` system calls, which widens the window by orders of magnitude and turns a rare race into the common case.

The fix

```
// The owner is captured HERE, while the kernel lock is still held, and the
// captured pointer -- not the member -- is the one dereferenced below.
thread* owner = owner_;

if ((owner != nullptr) && (boosted_prio_ > owner->priority_inherited ()))
{
    scheduler::uncritical_section sucs;

    // Still the owner? If it released the mutex while this section was open
    // there is nothing to inherit, and boosting it anyway would leave behind
    // an inherited priority that no later unlock() would ever clear.
    if (owner_ == owner)
    {
        owner->priority_inherited (boosted_prio_);
    }
}
```

Two properties restored: the dereference is of a pointer captured under the lock, and the boost is skipped if ownership changed — otherwise a thread that had just released the mutex would keep an inherited priority for ever.

`smp_test2` then passed **25 runs out of 25** at `OS_NCPU=4`. Both ARM QEMU suites were re-run against the changed kernel: 11 passed / 1 skipped / 0 failed on each, unchanged.

Why this is the whole justification for the port

A synthetic host cannot tell you how fast your RTOS is, and it cannot test your silicon. What it can do is **change the timing constants of the system by orders of magnitude while running the same kernel source**, so that windows too narrow to hit on hardware become the common case. That is what happened here, on the first day the port ran four CPUs.

19. Tips, tricks and traps

Reproducing a race

```
# 1. Turn the CPU count up. -DNCPU=8 on a 4-core laptop oversubscribes, which
#     lengthens every window in the port and the kernel.
cmake -S . -B /tmp/b8 -G Ninja -DBOARD=native -DNCPU=8 -DCMAKE_BUILD_TYPE=Release
cmake --build /tmp/b8 -j8

# 2. Run it many times and count. Intermittent means "count it", not "look at it".
cd /tmp/b8/test
pass=0; fail=0
for i in $(seq 1 50); do
    if timeout 60 ./smp_test2-host >/dev/null 2>&1; then pass=$((pass+1));
    else fail=$((fail+1)); fi
done; echo "pass=$pass fail=$fail"
```

A failure rate is data. "It sometimes crashes" is not, and neither is one run after a fix.

Other knobs that change the interleaving without changing the code:

knob	effect
<code>-DNCPU=n</code>	more CPUs than cores $\Rightarrow$ longer windows, more migration
<code>OS_INTEGER_SYSTICK_FREQUENCY_HZ</code>	a faster tick preempts more often; a slower one lets critical sections run to completion
<code>taskset -c 0,1</code>	confine the process to fewer cores; the opposite experiment
<code>nice -n 19</code> / a busy machine	starve CPUs unevenly
<code>-DCMAKE_BUILD_TYPE=Debug</code>	different inlining $\Rightarrow$ different windows (and it <i>has</i> moved failures)

Getting a line number out of a fault

```
# the report prints: pc 0x...8C6 (static 0x0000000000001A8C6)
addr2line -f -C -i -e test/build/test/smp_test2-host 0x1a8c6
```

- `-i` is essential at `-O2`; without it inlined frames are invisible.
- The **static** address is the one to pass. The raw `pc` includes the PIE load bias, which changes every run.
- The backtrace lines are already bias-corrected. The third entry is usually the signal trampoline and looks like garbage; the one after it is the faulting frame, and the one after *that* is who called it.

Debugging without a debugger

The standing rule in this workspace is no GDB in any test or run path. That is a hardware rule, but it turns out to be good practice here too, because attaching a debugger changes the timing that produces the bug. What to use instead:

want	tool
who called what	the built-in <code>backtrace</code> in the fault report
which syscalls	<code>strace -f -e trace=rt_sigprocmask,rt_sigaction,timer_settime ./t</code>
did it exit or crash	<code>strace -f -e trace=exit_group ./t</code> — an <code>exit_group(0)</code> with no output is a buffering bug, not a hang
is it spinning or blocked	<code>perf top -p \$(pgrep -f smp_test2-host)</code> ; a klock spin shows as <code>__atomic_exchange</code>
is the tick arriving	<code>strace -f -e 'trace=!all' -e signal=all ./t</code>
an invariant	a tripwire: check it, <code>write(2)</code> a message, <code>_exit(N)</code> with a distinct N

A tripwire beats a print: it fires once, at the moment the invariant breaks, and it does not perturb timing the way a per-switch `printf` does. Two of them are described in §7.6.

Things that will bite you

`printf` in a handler. Use `uart::uart1` or `write(2)`. `stdio` takes a lock; the lock may be held by a `µOS++` thread that is not running.

stdio buffering at `_exit`. To a pipe, `stdout` is fully buffered. If a test prints with `printf` and ends with `_exit`, the output vanishes. `hw_result` flushes; anything else must too.

`sigaction` is a function name as well as a struct tag. `sigaction sa;` does not compile. Write `struct sigaction sa;`.

`SIGRTMIN` is not a constant. In glibc it expands to `__libc_current_sigrtmin()`. It cannot appear in a `constexpr`, a `case` label or an array bound. The port wraps both signal numbers in inline functions.

`_XOPEN_SOURCE` must be global. Without it `ucontext.h` declares different things and you get a miscompile, not an error. The port `#error`s if it is missing.

Install the per-core idle threads before releasing the cores. `smp_install_boot_threads()` then `smp::start_secondary_cores()`, never the reverse. A released CPU enters `reschedule()` at once, and with no idle thread of its own it finds nothing to run and aborts with `!!! no ready thread and no idle thread on CPU 1 !!!`.

A sleeping thread tends to come back on CPU 0, because only CPU 0 calls `os_systick_handler()` and therefore sees it become runnable first. If a test of yours reports all its work on core 0, check whether its threads are CPU-bound or clock-bound before suspecting the scheduler. Four CPU-bound threads split 40/40/40/40 across four CPUs; the same four with a `sleep_for()` in the loop report core 0 every time. Both are correct.

Never cache a CPU index across a switch point. After `swapcontext()`, `sigsuspend()`, or anything that can block, re-read `port_cpu_id()`. The same goes for `errno` and for anything `thread_local`.

Never hold the kernel lock across a switch. `reschedule()` checks and defers; if you add a new path into `switch_stacks()`, it must check too.

Release the kernel lock owner-first. §6.4. The failure mode is a total silent freeze.

`getcontext()` **snapshots the caller's signal mask.** Any new context creation path must set `uc_sigmask` explicitly. §5.

A blank `-DNCPU=1` build is a *different kernel*. It is the non-SMP branch. Build and run `mutex-stress` after touching anything in `os-core.cpp`.

Rebuild the port you are about to test. A shared test source is still several separate binaries; this project has already lost an afternoon to testing an un-rebuilt target.

QEMU and host suites do not mix. Running the host suite while a four-core QEMU suite runs starves both. One suite at a time.

Reading a log

line	means
<code>rtos::memory::init_once_default_resource()</code>	very early; before the port's startup
<code>malloc_memory_resource() @... malloc</code>	the kernel's default resource, still installed
<code>first_fit_top(0x..., 33554432) @... app-heap</code>	the port's free store took over — if this is missing, <code>os_startup_initialize_hardware()</code> did not run
<code>estd::pmr::set_default_resource(...)</code>	...and became the default
<code>scheduler::start()</code>	the last line printed from <code>main()</code> ; everything after is a μ OS+ thread
<code>join: c1=3 c2=3 c3=3</code>	every secondary CPU reached <code>g_core_stage == 3</code>
<code>[LED on]</code> / <code>[LED off]</code>	the board's LED, printed only on change
<code>[cN]</code> prefixes	which CPU that thread was on when it printed

Adding something

A new test: create `test/native/<name>/` with at least one `.cpp`. That is all — the loop globs directories. Add a line to `tests.cmake` only if it needs a knob.

A new board (say, macOS): create `test/boards/<id>/` with a `board.cmake`, the four headers, `src/{heap,smp}.cpp` and a `run.sh`, plus `test/<id>/` for its applications. Nothing in the port's `CMakeLists.txt` changes. The work is in `host_cpu.cpp`: `SIGEV_THREAD_ID` does not exist on macOS, so the tick has to become a `kqueue` timer or a dispatch source, and `swapcontext` is deprecated there.

A new SD back-end: a new file under `micro-os-plus-iii-devices/soc/<id>/`, a new target, a new `#elif` in `sd.hpp`. Nothing existing changes — that is the contract.

20. Honest limits

Absolute timings mean nothing. `smp-mat-test` reports a 0.516× "speedup" for its parallel solver. Four host threads with microsecond context switches on an oversubscribed machine are not four Cortex-A53s. Use the port for correctness and interleaving, never for performance numbers.

The tick drifts under load. Once `OS_NCPU` exceeds the host's core count, timer deliveries queue and arrive in bursts. Kernel time stays self-consistent; wall time does not match it. `board-contract.cpp` warns above 16.

Thread-per-CPU is not thread-per-thread. A host debugger sees `OS_NCPU` threads, not one per μOS++ thread, and `swapcontext` confuses its unwinder. The port's own backtrace works because it is taken from the frame that faulted.

Sanitizers do not work yet. ASan and TSan both have opinions about `swapcontext` and about a signal handler that never returns to where it was raised. Making the port usable under them is its own piece of work — and it is the one thing a synthetic host ought eventually to be best at. See §22.

Wake-ups are biased towards CPU 0. Only CPU 0 advances the kernel clock — the BCM2837 arrangement this port copies — so a thread released by a timer, a `sleep_for()` or a timeout is usually re-picked by CPU 0 before any other CPU's tick comes round. Work that is CPU-bound spreads evenly; work that is clock-bound or I/O-bound does not. This is why `smp-pipeline-test` reports `Produced by Core: c0=3275 c1=0 c2=0 c3=0` and is not a defect in either the test or the port. Distributing the clock across CPUs would change it, and would also stop the port resembling the silicon it stands in for.

No USB, no real SD, no GPIO. The board does not claim `usb-device`, the SD card is a file, and the LED is a line of text. Three tests are not carried for exactly this reason.

`OS_NCPU=1` is a different kernel branch, not a degenerate case of the SMP one, and only `mutex-stress` runs there.

21. Decisions, and what was rejected

decision	chosen	rejected
what a CPU is	a host thread	a host thread per <code>μOS++</code> thread
preemption	preemptive, switch inside the handler	cooperative (upstream), or switch after <code>sigreturn</code>
migration	allowed; native TLS banned	pin <code>μOS++</code> threads to a CPU
the tick	per-CPU <code>timer_create</code> + <code>SIGEV_THREAD_ID</code>	<code>setitimer(ITIMER_REAL)</code>
masking	<code>pthread_sigmask</code>	<code>sigprocmask</code>
the kernel lock	raw atomic spin	<code>pthread_mutex</code>
the publish	deferred to the next arrival on this CPU	publish before <code>swapcontext</code>
the free store	<code>μOS++</code> <code>first_fit_top</code>	glibc <code>malloc</code>
tick handler stack	the thread's own	<code>sigaltstack</code>
<code>os-decls.h</code>	the port's own	<code>micro-os-plus::port-smp-decls</code>
SD	a third back-end	<code>#ifdef</code> inside the existing ones

One host thread per `μOS++` thread, with `OS_NCPU` run-tokens limiting concurrency, was the main alternative. It would give true preemption for free and much better debugger and sanitizer behaviour. It was rejected because it discards the working `ucontext` machinery and, more importantly, because it does not express the intended model: in it a host thread is a *thread*, and the whole point of D12 is that a host thread is a **CPU**. The kernel would then be exercised through a scheduler that is not its own.

Pinning `μOS++` threads to CPUs would have made native TLS safe and removed §7 entirely. It was rejected because it removes the thing worth testing: without migration there is no handover, no `stack_ptr` question, and `smp_test4` — load balancing — becomes meaningless.

Switching after `sigreturn` rather than inside the handler would be more conventional, but it means a switch can only ever happen at a point the thread chose, which is upstream's cooperative limitation wearing a timer.

`pthread_mutex` **for the kernel lock** is not merely slower; it is not async-signal-safe, and this lock is taken from signal handlers.

22. Reference tables

Files

file	lines	contents
<code>include/cmsis-plus/rtos/port/os-c-decls.h</code>	93	types, the thread context, the ucontext macro block
<code>include/cmsis-plus/rtos/port/os-decls.h</code>	191	<code>irq_set</code> , <code>_in_isr[]</code> , <code>lock_state[]</code> , the two locks, signal numbers, stack sizes
<code>include/cmsis-plus/rtos/port/os-inlines.h</code>	252	CPU id, critical sections, the raw locks
<code>include/host_cpu.hpp</code>	97	the CPU model's interface; <code>current_thread(cpu)</code>
<code>include/hw_result.hpp</code>	53	the verdict as an exit status
<code>src/rtos/os-core.cpp</code>	597	the port's half of the scheduler
<code>src/host_cpu.cpp</code>	413	handlers, ticks, IPI, bring-up, the publish slots
<code>src/exception_handler.cpp</code>	227	fault handlers and the report
<code>src/free-store.cpp</code>	133	<code>os_startup_initialize_free_store()</code> and the weak hooks
<code>src/board-contract.cpp</code>	43	<code>#error</code> s only
<code>src/diag/trace-posix.cpp</code>	80	upstream, unchanged
<code>test/boards/native/src/heap.cpp</code>	58	32 MiB <code>.bss</code> block, two ELF labels
<code>test/boards/native/src/smp.cpp</code>	32	<code>g_core_stage[]</code> , <code>start_secondary_cores()</code>

Environment variables

variable	default	used by
<code>BOARD</code>	<code>native</code>	<code>test/run.sh</code>
<code>BUILD</code>	<code>test/build</code>	<code>test/boards/native/run.sh</code>
<code>UOS_RUN_ONLY</code>	—	<code>test_smpl/run-host.sh</code>
<code>UOS_SMP_DIR</code>	the sibling	the board runner, CMake
<code>UOS_DEVICES_DIR</code>	the sibling	CMake
<code>UOS_SD_IMAGE</code>	<code>disk.img</code>	the host-file SD back-end
<code>UOS_SD_SECTORS</code>	<code>65536</code> (32 MiB)	the host-file SD back-end

CMake options

option	default	
<code>-DBOARD=</code>	<code>native</code>	must name a <code>test/boards/<id>/board.cmake</code>
<code>-DNCPU=</code>	<code>4</code>	host threads acting as CPUs
<code>-DUOS_SMP_DIR=</code>	sibling	the kernel working copy
<code>-DUOS_DEVICES_DIR=</code>	sibling	the devices working copy
<code>-DCMAKE_BUILD_TYPE=</code>	—	<code>Release</code> for the suite; <code>Debug</code> shifts the windows

Signals

signal	meaning	handler
<code>SIGRTMIN</code>	this CPU's tick	<code>tick_handler</code> — no <code>SA_ONSTACK</code>
<code>SIGRTMIN+1</code>	IPI	<code>ipi_handler</code> — no <code>SA_ONSTACK</code>
<code>SIGSEGV</code> <code>SIGBUS</code> <code>SIGFPE</code> <code>SIGILL</code>	a synchronous fault	<code>fault_handler</code> — <code>SA_ONSTACK</code>

Where things live

port	<code>micro-os-plus-iii-posix-arch</code>
kernel	<code>micro-os-plus-iii-smp</code>
devices	<code>micro-os-plus-iii-devices</code>
the SD back-end	<code>micro-os-plus-iii-devices/soc/native/</code>
the shared runner	<code>micro-os-plus-iii-smp/test_smpl/run-host.sh</code>
the design decision	<code>docs/specs/2026-09-20-...-unification-design.md</code> §7.6, D12
the sibling ports	<code>cortexm-port.md</code> , <code>tests-in-aarch32-aarch64.md</code>

23. Worked examples

Everything below is complete and compiles as written, against this port at `OS_NCPU=4`. Paths are relative to `micro-os-plus-iii-posix-arch/`.

23.1 The smallest application that runs

Create one directory; that is the whole build change.

```
test/native/hello/main.cpp
```

```

/*
 * hello – the smallest µOS++ III SMP application on the POSIX host.
 *
 * It is deliberately written the way the carried tests are, because that is
 * the shape that also compiles on a Raspberry Pi: the application owns main(),
 * brings up its own console in os_startup_initialize_hardware(), and installs
 * the free store from the board's heap labels. On this port none of that is
 * strictly required -- the kernel's weak main() would do (see 23.2) -- but
 * writing it this way means the file can be dropped into an ARM board's test
 * directory unchanged.
 */
#include <cmsis-plus/rtos/os.h>

#include <uart.hpp>           // the board's console: uart::uart1
#include <hw_result.hpp>      // the verdict, as an exit status
#include <exception_handler.hpp> // exception::init()

using namespace os::rtos;

extern "C" {

// Provided by the board: test/boards/native/src/heap.cpp.
extern char __heap_start[];
extern char __heap_end[];

void os_startup_initialize_hardware_early (void) { }

void os_startup_initialize_hardware (void)
{
    uart::uart1.init ();
    uart::uart1 << "\n+== " PORT_BANNER_SHORT " hello (" PORT_BANNER_ISA ") ==+\n\n";

    os_startup_initialize_free_store (
        __heap_start, static_cast<std::size_t> (__heap_end - __heap_start));

    exception::init ();
}

[[noreturn]] static void main_trampoline (void)
{
    std::exit (os_main (0, nullptr));
}

extern void os_startup_create_thread_idle (void);

```

```

extern os::rtos::thread* os_main_thread;

int main (int, char*[])
{
    scheduler::initialize ();           // the port's startup runs inside this

    static thread::stack::element_t main_stack[8192];    // 8192 × 8 B = 64 KiB
    thread::attributes attr = thread::initializer;
    attr.th_stack_address    = main_stack;
    attr.th_stack_size_bytes = sizeof (main_stack);
    static thread main_thread { "main",
        reinterpret_cast<thread::func_t> (main_trampoline), nullptr, attr };
    os_main_thread = &main_thread;

    os_startup_create_thread_idle ();
    scheduler::start ();                // does not return
    return 0;
}

} /* extern "C" */

extern "C" unsigned port_cpu_id (void);

int os_main (int, char*[])
{
    for (int i = 0; i < 5; ++i)
    {
        uart::uart1 << "hello " << i
            << " from core " << static_cast<int> (port_cpu_id ())
            << " at " << static_cast<int> (sysclock.now ()) << " ms\n";
        sysclock.sleep_for (200);
    }

    uart::uart1 << "\nRESULT: PASS\n";
    hw_result::ok ();                  // flushes, then _exit(0)
}

```

Build and run it — no CMake edit at all, because the loop globs directories:

```

$ cmake -S . -B test/build -G Ninja -DBOARD=native -DCMAKE_BUILD_TYPE=Release
-- µOS++ III POSIX tests: 11 targets, board=native, OS_NCPU=4, ...
$ cmake --build test/build -j8
$ BOARD=native test/run.sh hello
hello
rtos::memory::init_once_default_resource()
malloc_memory_resource() @0x562f7c069b00 malloc

+== native hello (x86-64) ==+

first_fit_top(0x562f7c0ac2a0,33554432) @0x562f7c0ac230 app-heap
out_of_memory_handler(0x562f7c04a950) @0x562f7c0ac230 app-heap
std::pmr::set_default_resource(0x562f7c0ac230)
scheduler::start()
hello 0 from core 0 at 0 ms
hello 1 from core 0 at 200 ms
hello 2 from core 0 at 400 ms
hello 3 from core 0 at 600 ms
hello 4 from core 0 at 800 ms

RESULT: PASS
PASS

host suite: 1 passed, 0 skipped, 0 failed

```

Read the first four lines as a checklist, because they are the ones that go missing when something is wrong (§19, *Reading a log*): `first_fit_top(...)` `app-heap` says `os_startup_initialize_hardware()` ran and the port's free store replaced the kernel's `malloc` resource; `scheduler::start()` is the last line printed from `main()`.

Every line says core 0, and that is correct here — for two reasons, and both are worth understanding before concluding anything about the port.

First, this application builds at `OS_NCPU=4` — four CPUs *exist* — but it never calls `smp::start_secondary_cores()`, so CPUs 1 to 3 have no host thread and no tick. A thread cannot migrate to a core that has not been released, exactly as on a Raspberry Pi whose secondary cores are still parked in the spin table.

Second, and this one survives releasing them: **a thread that spends its life in `sleep_for()` tends to stay on CPU 0**. It is made ready by `os_systick_handler()`, and only CPU 0 calls that (§8.4, the BCM2837 arrangement); CPU 0 therefore sees it become runnable first and re-picks it before any other CPU's tick comes round. Adding the release calls to this example changes nothing about its output:

```
hello 0 from core 0 at 0 ms
hello 1 from core 0 at 200 ms
...
```

That is not the scheduler failing to balance — it is one runnable thread and one CPU that always hears about it first. §23.3 shows what happens when there is actual work to spread.

23.2 The same thing, the upstream way

If a file never has to compile for bare metal, the kernel's own weak `main()` will do everything above. The whole application is then:

```
#include <cmsis-plus/rtos/os.h>
#include <hw_result.hpp>
#include <cstdio>

extern "C" unsigned port_cpu_id (void);

int os_main (int, char*[])
{
    std::printf ("hello from core %u\n", port_cpu_id ());
    std::printf ("\nRESULT: PASS\n");
    hw_result::ok ();          // the fflush() inside matters here — see §16
}
```

That is how `mutex-stress` is built. The kernel's `main()` prints the version banner and `port::scheduler::greeting()`, calls `scheduler::initialize()`, creates `main` and `idle`, and starts the scheduler. The application free store is *not* installed in this path, so the kernel's `malloc` resource stays — fine for a test, wrong for anything that wants the deterministic heap of §11.

23.3 Threads on several CPUs, with a mutex

The shape of `smp_test2`, reduced to what matters.

```

#include <cmsis-plus/rtos/os.h>
#include <uart.hpp>
#include <smp.hpp>           // smp::start_secondary_cores(), g_core_stage[]
#include <test-smp-boot.hpp> // smp_install_boot_threads(), test_wait_secondaries()
#include <hw_result.hpp>

using namespace os::rtos;
extern "C" unsigned port_cpu_id (void);

namespace {
    mutex    g_mutex { "cnt" };
    unsigned g_counter = 0;

    constexpr unsigned kIters = 2000;

    // One stack per worker. element_t is uint64_t, so 4096 elements = 32 KiB,
    // which is OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES – the minimum on this port
    // because a signal frame lands on the thread's own stack (§8.3).
    thread::stack::element_t wstack[OS_NCPU][4096];

    void* worker (void*)
    {
        for (unsigned i = 0; i < kIters; ++i)
        {
            g_mutex.lock ();
            g_counter = g_counter + 1;           // read-modify-write under the lock
            g_mutex.unlock ();

            if ((i & 0x7F) == 0)
                this_thread::yield ();           // give the other CPUs a chance
        }

        g_mutex.lock ();
        uart::uart1 << "[c" << static_cast<int> (port_cpu_id ())
                     << "]" done, counter=" << g_counter << "\n";
        g_mutex.unlock ();
        return nullptr;
    }
}

int os_main (int, char*[])
{
    // 1. Give every secondary CPU an idle thread, BEFORE releasing it. The
    //    kernel supplies CPU 0's; these are CPUs 1..N-1's.

```

```

//
//   THE ORDER IS NOT A STYLE CHOICE. A released CPU enters reschedule()
//   immediately, and if its idle thread does not exist yet it finds
//   nothing at all to run:
//
//       !!! no ready thread and no idle thread on CPU 1 !!!
//       Aborted (core dumped)
//
//   Every carried test does these two in this order for this reason.
smp_install_boot_threads ();

// 2. Release the other CPUs. On a Pi this is a spin table and a mailbox
//   doorbell; here it is pthread_create. Same call, same place.
smp::start_secondary_cores ();

// 3. Wait for them to reach stage 3, exactly as on silicon.
test_wait_secondaries (1000);

// 4. One worker per CPU. No affinity is set anywhere: the scheduler is free
//   to run any of them on any CPU, and will.
static char name[OS_NCPU][8];
static thread* w[OS_NCPU];
for (unsigned c = 0; c < OS_NCPU; ++c)
{
    std::snprintf (name[c], sizeof (name[c]), "%u", c);
    thread::attributes a = thread::initializer;
    a.th_stack_address = wstack[c];
    a.th_stack_size_bytes = sizeof (wstack[c]);
    w[c] = new thread { name[c], worker, nullptr, a };
}

for (unsigned c = 0; c < OS_NCPU; ++c)
    w[c]->join ();

const unsigned expected = OS_NCPU * kIters;
uart::uart1 << "counter=" << g_counter << " expected=" << expected << "\n";
uart::uart1 << "\nRESULT: " << ((g_counter == expected) ? "PASS" : "FAIL") << "\n";

(g_counter == expected) ? hw_result::ok () : hw_result::fail ();
}

```

Proof that this is really SMP

The shortest demonstration, measured rather than argued. Four **CPU-bound** threads — no `sleep_for()`, so nothing funnels their wake-ups through CPU 0 — each reporting `port_cpu_id()` forty times:

```
static void* busy (void*)
{
    for (int i = 0; i < 40; ++i)
    {
        volatile unsigned long x = 0;
        for (unsigned long k = 0; k < 3000000UL; ++k) x += k;
        uart::uart1 << "  busy on core " << static_cast<int> (port_cpu_id ()) << "\n";
    }
    return nullptr;
}
```

```
$ ./test/build/test/mytest-host | grep 'busy on core' | sort | uniq -c
    40  busy on core 0
    40  busy on core 1
    40  busy on core 2
    40  busy on core 3
```

Four threads, four CPUs, an even split, and no affinity set anywhere. Replace the spin with `sysclock.sleep_for(50)` and the same program prints *core 0* every time — for the reason in §23.1, not because the scheduler stopped working.

`smp_test4` makes the stronger statement, because its eight workers are more numerous than the CPUs and it tallies where each one ran:

```
-- reporter on c0 t=9007ms --
w0 [ c0=3297 c1=3794 c2=3810 c3=3683 ]
w1 [ c0=3278 c1=3646 c2=3927 c3=3706 ]
...
```

Every worker has run on every CPU, thousands of times each. That is migration, and it is the deferred publish of §7 working ten thousand times a second.

Where the helpers come from

`smp_install_boot_threads()` and `test_wait_secondaries()` come from `test/native/src/test-smp-boot.cpp`, which the build adds to every test on the board automatically — one copy, not one per test. That file is byte-identical in purpose to the ARM boards' copy; its only POSIX-

specific line is the idle body's `port::scheduler::wait_for_interrupt()`, which is the kernel's own port API and is `sigsuspend()` here, `dsb sy; wfi` on ARMv8-A.

23.4 Using the SD card

`sd::SdCard` is the same class on all three back-ends. Nothing in an application selects one — the board's `UOS_BOARD_DEVICES` does.

```
#include <sd.hpp>           // dispatches on SD_BACKEND_*; here: soc/native
#include <flatfs.hpp>

namespace {
    sd::SdCard    g_card;
    flatfs::Fs    g_fs;
    std::uint8_t  g_sector[512];
}

bool storage_init ()
{
    if (!g_card.init ())
    {
        uart::uart1 << "SD init failed: " << g_card.error () << "\n";
        return false;
    }

    uart::uart1 << "SD ready: " << g_card.sector_count ()
                << " sectors (~" << (g_card.capacity_bytes () >> 20) << " MiB)\n";

    // HW_BUILD is not defined on this port ($14), so the tests take this branch:
    // format a flatfs volume on the raw device rather than mounting a FAT32
    // boot partition that a host file does not have.
    if (!g_fs.format (g_card))
        return false;

    return true;
}
```

and to select the file it lives in:

```
$ UOS_SD_IMAGE=/tmp/card.img UOS_SD_SECTORS=131072 ./test/build/test/mytest-host
```

Declare the dependency once, in `test/native/tests.cmake`:

```
set (BOARD_TEST_NEED_DEVICES smp-num-test smp-pipeline-test mytest)
```

which links `UOS_BOARD_DEVICES` — `micro-os-plus::devices-hostfile` — for those tests only. The image is created if missing and never deleted, so after the run:

```
$ ls -l /tmp/card.img
-rw-r--r-- 1 dan dan 67108864 ... /tmp/card.img
$ xxd -l 64 /tmp/card.img
```

That is a real advantage of this back-end over the two silicon ones, not a compromise: the volume a failing test left behind can be examined with ordinary tools.

23.5 A test that runs at a different CPU count

`board_test_ncpu()` is per test, and `uos_add_app()` turns it into both `OS_NCPU` and `OS_USE_SMP_SCHEDULER`. To add a single-core test beside the four-core ones:

```
# test/native/tests.cmake
function (board_test_ncpu _app _out)
  if (_app STREQUAL "mutex-stress" OR _app STREQUAL "mytest-sc")
    set (${_out} 1 PARENT_SCOPE)          # non-SMP kernel branch
  elseif (_app STREQUAL "mytest-16")
    set (${_out} 16 PARENT_SCOPE)         # oversubscribe on purpose
  else ()
    set (${_out} "${UOS_BOARD_NCPU}" PARENT_SCOPE)
  endif ()
endfunction ()

# A single-core test must not get the board's SMP boot helper: that file
# installs per-core idle threads through scheduler::os_idle_thread_core[],
# which does not exist without OS_USE_SMP_SCHEDULER.
set (BOARD_TEST_SELF_CONTAINED mutex-stress mytest-sc)
```

Verify it reached the compiler rather than trusting the CMake:

```
$ grep -o 'OS_NCPU=[0-9]*' test/build/build.ninja | sort | uniq -c
    47 OS_NCPU=1
   427 OS_NCPU=4
$ grep -c OS_USE_SMP_SCHEDULER \
    test/build/test/CMakeFiles/mutex-stress-host.dir/*.d 2>/dev/null
0
```

23.6 A tripwire

When an invariant is in doubt, assert it where it would break and leave nothing behind. This is the "one CPU per thread" check that was compiled into `switch_stacks()` during bring-up (§7.6) — it never fired, which is how the claim point was shown to be sound.

```
#if defined(UOS_SMP_TRIPWIRE)
    // The property the whole deferred-publish design exists to keep:
    // a thread is current on at most one CPU.
    for (unsigned c = 0; c < OS_NCPU; ++c)
    {
        if (c != cpu && rtos::scheduler::current_thread[c] == new_thread)
        {
            char buf[200];
            int n = snprintf (buf, sizeof (buf),
                "\n!!! TRIPWIRE: cpu%u claimed \"%s\", already current on cpu%u\n",
                cpu, new_thread->name (), c);
            ssize_t r = ::write (2, buf, (size_t) n);    // write(2), not printf
            (void) r;
            ::_exit (4);                                // a distinct exit code
        }
    }
#endif
```

```
$ cmake -S . -B /tmp/tw -G Ninja -DBOARD=native -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_CXX_FLAGS=-DUOS_SMP_TRIPWIRE
$ cmake --build /tmp/tw -j8
$ cd /tmp/tw/test
$ fired=0; for i in $(seq 1 50); do ./smp_test2-host >/dev/null 2>&1
    [ $? -eq 4 ] && fired=$((fired+1)); done; echo "tripwire fired $fired/50"
tripwire fired 0/50
```

Three rules that make a tripwire useful rather than misleading:

- `write(2)`, never `printf`. It may fire inside a signal handler.
- **A distinct exit code**, so a loop can count it without parsing output.
- **Take it out again.** A tripwire that survives into a commit is a `printf` with extra steps, and the added call *changes timing* — one of these perturbed a 3-in-5 failure into 6 clean runs, which proved nothing at all.

23.7 Counting an intermittent failure

Never conclude from one run. This is the loop that turned "smp_test2 sometimes crashes" into "21 out of 25", and then into "0 out of 25".

```
#!/usr/bin/env bash
# fail-rate <app> [runs] [ncpu]
app="${1:?}"; runs="${2:-25}"; ncpu="${3:-4}"
build="/tmp/fr-$ncpu"

cmake -S . -B "$build" -G Ninja -DBOARD=native -DNCPU="$ncpu" \
      -DCMAKE_BUILD_TYPE=Release >/dev/null
cmake --build "$build" -j8 >/dev/null

pass=0; fail=0
for i in $(seq 1 "$runs"); do
    if timeout 120 "$build/test/$app-host" >"/tmp/$app.$i.log" 2>&1
    then pass=$((pass+1)); rm -f "/tmp/$app.$i.log"
    else fail=$((fail+1)); fi
done
echo "$app at NCPU=$ncpu: pass=$pass fail=$fail"
```

```
$ ./fail-rate smp_test2 25 4
smp_test2 at NCPU=4: pass=25 fail=0
$ ./fail-rate smp_test2 25 8
smp_test2 at NCPU=8: pass=25 fail=0
```

The kept logs of the failing runs are the evidence; the ones that passed are deleted so the directory holds only what matters.

23.8 From a fault to a line

The complete recipe, from the report in §12.

```
$ ./test/build/test/smp_test2-host
!!! FATAL EXCEPTION on CPU 1 -- signal Segmentation fault at 0x...C2,
                                pc 0x000055AF745AA8C6 (static 0x000000000001A8C6)

cpu0: idle
cpu1: w1
cpu2: idle2
cpu3: w3
klock: owner=0x...3 depth=0x...0 lock=0x...0
backtrace (static):
    0x000000000001E37F      ← port_fatal_exception
    0x000000000001E608      ← fault_handler
    0x000029AFC44AE6F0      ← the signal trampoline (ignore)
    0x000000000001A8C6      ← the faulting frame
    0x00000000000172DD      ← its caller
    0x0000000000017695      ← and its caller
```

```
$ addr2line -f -C -i -e test/build/test/smp_test2-host 0x1a8c6 0x172dd 0x17695
os::rtos::thread::priority_inherited(unsigned char)
os::rtos::mutex::internal_try_lock_(os::rtos::thread*)
os::rtos::mutex::lock()
```

and, when the line is not enough, the instruction itself:

```
$ objdump -d --start-address=0x1a8c0 --stop-address=0x1a8d0 \
    test/build/test/smp_test2-host
1a8c6: 40 38 b3 c2 00 00 00    cmp    %sil,0xc2(%rbx)
```

`%rbx` is `this`; `0xc2` is the offset of `prio_inherited_`; the faulting address was `0xc2`. Therefore `this == nullptr` — not a corrupted pointer, a null one. §18 continues from here.

Three things to remember:

- Pass the **static** address, not the raw `pc`. The raw one includes the PIE load bias and changes every run.
- `-i` is not optional at `-O2`. Without it every inlined frame is invisible and you get `???`.
- The backtrace entries are already bias-corrected by the reporter.

23.9 A second host board

The directory structure is ready; the work is in `host_cpu.cpp`. A macOS board would be:

```

test/boards/macos/
├─ board.cmake          # a copy of native's, with its own PORT_GREETING
├─ include/{uart,led,timer_arm,smp}.hpp  # identical: write(2), CLOCK_MONOTONIC
├─ src/{heap,smp}.cpp    # identical except the .section syntax
├─ run.sh                # identical
test/macos/              # its applications

```

`board.cmake` in full:

```

set (UOS_BOARD_SRC_DIR "${CMAKE_CURRENT_LIST_DIR}")
set (UOS_BOARD_FLAGS "")
set (NCPU 4 CACHE STRING "Host threads acting as CPUs")
set (UOS_BOARD_NCPU ${NCPU})
set (UOS_BOARD_CAPS smp sdcard led)
set (UOS_BOARD_LIBS "")
set (UOS_BOARD_DEVICES micro-os-plus::devices-hostfile)
set (UOS_BOARD_DEFINES "PORT_GREETING=\"µOS++ macOS synthetic host\"")

```

and that is all CMake needs — the board is discovered by the presence of the file. What actually has to be written:

what	why
the tick	<code>SIGEV_THREAD_ID</code> does not exist. Use a <code>kqueue</code> <code>EVFILT_TIMER</code> per CPU, or a dispatch source, and raise the signal with <code>pthread_kill</code> on that CPU's own thread
<code>swapcontext</code>	deprecated on macOS. Either accept the warning, or build with <code>OS_INCLUDE_LIBUCONTEXT</code> , which the port already supports — <code>os-c-decls.h</code> carries upstream's macro block for exactly this
<code>g_fault_stack</code>	<code>sigaltstack</code> works, but <code>SIGSTKSZ</code> differs; the port already uses a fixed 64 KiB
<code>.section</code> syntax	Mach-O, not ELF: <code>heap.cpp</code> 's <code>#error</code> says so rather than emitting something wrong
<code>syscall(SYS_gettid)</code>	no equivalent; the tid is not needed once the timer is per-CPU by construction

Nothing in `src/rtos/os-core.cpp` should need to change. That file already guards on `defined(__APPLE__) || defined(__linux__)`.

23.10 What *not* to write

Four things that compile and are wrong on this port. Each one was an actual bug here.

```
// WRONG — the CPU index is stale after any switch point.
const unsigned cpu = port_cpu_id ();
sysclock.sleep_for (10);
do_something_with (cpu);           // we may be on another CPU now

// RIGHT
sysclock.sleep_for (10);
const unsigned cpu = port_cpu_id ();
do_something_with (cpu);
```

```
// WRONG — errno is host-thread local, and the thread may have moved.
int fd = ::open (path, O_RDONLY);
this_thread::yield ();
if (fd < 0) report (errno);        // whose errno?

// RIGHT — read it inside the section that made the call.
int fd = ::open (path, O_RDONLY);
int e = errno;
this_thread::yield ();
if (fd < 0) report (e);
```

```
// WRONG — thread_local is per host thread, i.e. per CPU, not per µOS++ thread.
thread_local int my_depth = 0;    // "my" is a lie the moment it migrates

// RIGHT — put it in the thread's own storage, or pass it.
```

```
// WRONG — printf from anything a signal handler can reach.
void my_isr_work () { std::printf ("tick %u\n", n); } // stdio takes a lock

// RIGHT
void my_isr_work () { uart::uart1 << "tick " << n << "\n"; } // write(2)
```

What is not done

- **rtos-apis**, upstream's larger API suite, at `OS_NCPU=1`. It needs FatFs and posix-io wired to the host.
- **Sanitizers** (§20). The highest-value remaining work on this port.
- **A second board**. It would be a different *host* — macOS, where `SIGEV_THREAD_ID` does not exist. The directory structure is ready for it; `host_cpu.cpp` is where the work is.

- **OS_NCPU=1** for the **SMP tests**. Not possible without editing them (`cpu_affinity()`), and not worth it.